



— CI / CD • PIPELINE TO OFFER

Гайд по перекасту в DevOps

Понятный путь от первых команд в терминале до
первого оффера — на позиции **от 250 000 ₽**.

BUILD

TEST

DEPLOY

OFFER

Linux

Docker

Kubernetes

CI / CD

Terraform

Ansible

deploy.sh — career@devops

BASH

```
$ kubectl apply -f career.yaml
deployment.apps/junior-devops created
$ kubectl rollout status deploy/career
Waiting for rollout... 3 of 3 ready
$ echo "offer accepted"
```

Для тех, кто хочет не просто учить
технологии, а **выйти на доход в IT**.



Виктор Шутов

@BOOOSTMENTOR • DevOps mentor

Этот гайд проведёт тебя от нуля до первого оффера

Без магии и «оффера за неделю». Гайд показывает **путь**: какие навыки собрать, что сделать руками, как проверить готовность и как подойти к первым собеседованиям — без хаоса.

ЭТОТ ГАЙД ДЛЯ ТЕБЯ, ЕСЛИ:



Сисадмин

Умеешь в Linux и сети, но контейнеры и K8s звучат пугающе.



Разработчик

Видел прод со стороны кода, читал CI-пайплайны, но не строил инфраструктуру сам.



Студент или начинающий

Хочешь войти в IT сразу в востребованную специализацию.



Уже проходишь DevOps-курс

Но не понимаешь, что важно на собесе, а что можно пропустить.

ЧТО ВНУТРИ

Не теория про «культуру DevOps». Конкретный путь: какие навыки нужны, в каком порядке их строить, что спрашивают на реальных собеседованиях в российских компаниях, как выглядит сильное резюме и какие проекты показывать.



Ориентир: позиции от 250 000 ₽ + для людей с серьёзной подготовкой. Конкретная цифра зависит от рынка, региона и компании.



ЗАЧЕМ ЭТО ВСЁ

DevOps — одно из самых востребованных и **высокооплачиваемых** направлений. Этот гайд — короткий маршрут к точке, где тебя начинают звать на собеседования.



Путь перехода — от основ до оффера

теория + руками

01

Почему DevOps, кто такие DevOps и SRE

Спрос на рынке, чем DevOps отличается от SRE, зона ответственности руками, карта навыков.

02

Что учить и в каком порядке

Приоритеты из реального банка вопросов. Почему Kubernetes — не точка входа.

03

Roadmap по неделям

Понедельный ориентир с проверками готовности на каждом этапе.

04

Базовый старт + путь запроса

Арендовать сервер, бэкенд за Nginx-gateway, DNS и HTTPS, база в приватной сети — как трафик доходит до сервиса.

05

Технологии по порядку — что сделать руками

Linux · сети · Docker · CI/CD · Terraform+Ansible. По каждой: что знать, как спрашивают, что проверяет интервьюер, задание руками, как в резюме, типичная ошибка.

06

Kubernetes + Helm

Простыми словами, архитектура кластера (схема), основные ресурсы и команды, Helm.

07

Capstone — от minikube к боевому кластеру

minikube → kubespray (3 ноды, боевая лаба) → fe+be из GitLab + Prometheus/Grafana/Loki + отказоустойчивость.

08

Собесы, ошибки, резюме

Реальные вопросы с эталонными ответами, 6 ошибок, формулировки слабо → сильно.

09

Программа школы и следующий шаг

Реальные стенды, troubleshooting-first, mock-интервью, карьерный трек.



Почему DevOps — и почему сейчас

DevOps — одна из немногих IT-специальностей, где спрос стабильно превышает предложение. Компании строят платформы, переезжают в Kubernetes, переходят на GitOps — и каждому переходу нужен человек, умеющий делать это руками.

Что это значит руками

Инженер пишет `CI/CD pipeline`, который соберёт образ, запустит в Harbor, задеплоит в Kubernetes через Helm и откатится автоматически, если readiness probe не прошла. Это его прямой продукт.

Почему сейчас хороший момент

Большинство компаний ищут не звёзд с 10 годами опыта, а людей, которые умеют чинить — и умеют объяснить, как они чинили. Человек с технической базой и правильной подготовкой может претендовать на позиции, которые раньше казались недостижимыми.

Что делает направление перспективным

- Высокий и стабильный спрос, в том числе на удалёнке.
- Навыки переносимы между компаниями — учишь один раз.
- Понятный пост: junior → middle → senior → SRE / platform.
- Видимый результат: пайплайн, который работает, — твоя заслуга.

КЛЮЧЕВАЯ МЫСЛЬ

Тебе не нужно быть сильным программистом. Нужно понимать, как системы работают вместе.

Это снижает порог входа для свитчеров с опытом в смежных областях — поддержка, тестирование, сисадминство.



ОШИБКА НОВИЧКА

Думать, что DevOps — это «поставить Kubernetes и готово». По данным банка 1165 реальных вопросов с собеседов в 38 компаниях, большинство провалов происходит не на K8s-вопросах, а на **Linux, сетях и troubleshooting**.



Кто такой DevOps — и кто такой SRE

Прежде чем нырять в инструменты — короткая сноска про роли. Эти слова постоянно путают, а на собесе любят спросить разницу.

DEVOPS

Не «эникейщик» и не «человек-кубернетес». DevOps — про то, чтобы путь **от кода до прода был быстрым и безопасным**. Снимает стену между разработкой и эксплуатацией.

- CI/CD — автоматическая сборка и доставка
- Инфраструктура как код (Terraform, Ansible)
- Контейнеры и оркестрация (Docker, Kubernetes)
- Эксплуатация и автоматизация рутины

SRE • SITE RELIABILITY ENGINEERING

Подход, который придумали в Google: **надёжность как инженерная задача**, а не «дежурный, который тушит пожары».

- **SLI / SLO** — сколько система обязана работать
- **Error budget** — сколько ей позволено ломаться
- **Борьба с toil** — ручную рутину убираем кодом
- Дежурства (on-call) и постмортемы без поиска виноватых



КАК ОТВЕЧАТЬ НА СОБЕСЕ

«DevOps — про **скорость и автоматизацию доставки**; SRE — про **надёжность через инженерные практики** (SLO, error budget, борьба с рутинной)». На практике в РФ вакансии часто пересекаются, инструменты одни и те же (Linux, Kubernetes, Prometheus, Terraform) — нередко это одна команда, а то и один человек.



Что делает DevOps-инженер день за днём

ЗАДАЧА • ЧТО ЭТО ЗНАЧИТ РУКАМИ

7 зон ответственности

ЗАДАЧА	ЧТО ЭТО ЗНАЧИТ РУКАМИ
CI/CD	Пайплайны lint → test → build → push → deploy; переиспользуемые шаблоны для 80+ сервисов
Kubernetes	Кластеры (kubeadm, Kubespray, managed), workloads, ingress, RBAC, NetworkPolicy, probe'ы
IaC	Terraform поднимает VM + сеть; Ansible раскладывает конфиги и сервисы
Observability	Prometheus + Grafana + Alertmanager + Loki. Алерт до того, как пожаловался пользователь
Инциденты	Диагностика по слоям: ping → порт → DNS → TLS → app. Корень — фикс — постмортем
Секреты	HashiCorp Vault: ротация, audit log, интеграция с K8s через JWT-auth
Безопасность	Trivy на образы, checkov на Terraform, NetworkPolicy, mTLS через Istio

JUNIOR VS MIDDLE — В ЧЁМ РАЗНИЦА

Junior знает **как запустить**. Middle знает **как не уронить и как починить**. Разница не в инструментах — она в мышлении: «что может пойти не так», «какой blast-radius», «как убедиться, что изменение применилось, а не просто прошло через pipeline».



СОВЕТ

На собесе тебя не спросят «знаешь ли ты Prometheus». Тебя спросят: «как ты добился, что MTTR снизился с 2 часов до 15 минут?» Ответ должен быть конкретным.



Карта навыков: 11 доменов

Они связаны строгим графом зависимостей: нельзя учить Kubernetes раньше, чем понимаешь сети — иначе ты не поймёшь, почему pod не видит pod.

ДОМЕН	УРОВЕНЬ	НА СОБЕСЕ СПРАШИВАЮТ
Linux	junior	inode, FD, cgroups, systemd, процессы
Сети	junior → mid	OSI, TCP, DNS, iptables, VPN, TLS
Troubleshooting	сквозной	«Сайт отдаёт 502 — что делаешь первым?»
Docker	junior	multi-stage, Kaniko, registry, диагностика
CI/CD	junior → mid	Pipeline design, стратегии деплоя, quality gates
IaC	junior → mid	Terraform state, plan vs apply, Ansible поли
Kubernetes	middle	286 вопросов в банке — главная тема собеседа
Helm	middle	Chart structure, values, hooks, rollback
Observability	middle	Prometheus, Grafana, Loki, Alertmanager
GitOps	middle	ArgoCD, app-of-apps, sync-waves, drift
DevSecOps	сквозной	Vault, RBAC, NetworkPolicy, Trivy, checkov



ВАЖНО

Kubernetes намеренно в середине списка, не в начале. По данным 177 реальных собеседований — большинство провалов происходит на Linux, сетях и troubleshooting. Именно там можно набрать очков, если подготовлен лучше остальных.



Что учить и в каком порядке — честно

Интернет полон советов «начни с Kubernetes». Не начинай с Kubernetes. Когда pod не видит другой pod, проблема может быть в DNS CoreDNS, NetworkPolicy, kube-proxy iptables или CNI. Чтобы локализовать корень — нужно понимать сети. Kubernetes — верхний слой, а не точка входа.

Приоритеты по темам (из 1165 реальных вопросов)

286 вопр.

Kubernetes — главная тема. Учи глубоко: scheduler, requests/limits, rolling update, PDB.

115 вопр.

Сети — OSI, путь пакета, TCP, DNS, iptables, балансировка. «Что при curl yandex.ru?»

81 вопр.

Linux — inode, процессы, cgroups, systemd. Ловушки типа «место есть, запись падает».

84 вопр.

CI/CD — pipeline design, стратегии деплоя, canary vs blue-green, quality gates.

47 вопр.

IaC — Terraform state, plan vs apply, drift, Ansible handlers, blast-radius.



ДЕНЬГИ • КАРЬЕРА

Граница Junior-трудоустраиваемости: после Linux + сети + troubleshooting + Docker + CI/CD + IaC. **Kubernetes — это уже Middle.** Не пытайся закрыть всё до первого оффера.

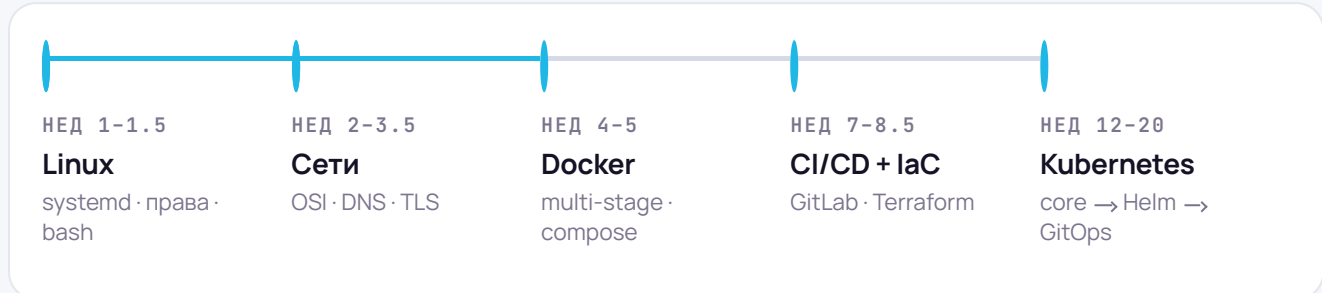
Чего не учить в начале

- Service mesh (Istio) — нужен только когда понимаешь K8s networking
- ArgoCD / GitOps — надстройка над работающим кластером
- Kafka, PostgreSQL HA — специфика, придёт в процессе работы



Roadmap: путь от «хочу» до первого оффера

Ориентир для ~10 ч/нед. Реальный темп зависит от базы и интенсивности. Эти оценки — стартовая точка, не обещание.



Проверки готовности по этапу

Linux

inode-exhaustion и D-state
Объяснишь: df vs du (открытый FD), load avg выше ядер при простаивающем CPU.

Сети

Локализация обрыва по слоям
ICMP → порт → DNS → TLS → application. Методично, без угадывания.

Docker

multi-stage + scratch
Go-бинарь из scratch, musl vs glibc, почему не :latest в CI.

CI/CD

Pipeline с stages, rules, needs
Разобрать чужой pipeline и найти проблемы.

K8s

Control-plane, RBAC, probe'ы
Архитектура, Deployment vs ReplicaSet, requests vs limits.

Навыки по уровням

JUNIOR **база**

- Linux, bash, git, сети
- Docker, docker-compose, CI/CD
- IaC: Terraform + Ansible

MIDDLE **прод**

- Kubernetes, Helm, GitOps
- Observability, Vault
- Облако: Yandex Cloud / AWS

SENIOR **масштаб**

- SRE, observability в масштабе
- FinOps, безопасность
- Platform engineering



ВАЖНО

Калибруй темп под себя после первых 2-3 недель. Главный критерий прогресса — не пройденные темы, а **задачи, которые ты можешь решить руками**.



Технологии по порядку — не теория, а руками

Дальше — разбор каждой технологии по одной и той же схеме. Сначала простыми словами «что это», потом — что реально сделать руками. Не «прочитать про Kubernetes», а **поднять, сломать, починить**.

СХЕМА РАЗБОРА • ОДИНАКОВАЯ ДЛЯ КАЖДОЙ ТЕМЫ

- **Что это / что знать** — коротко и по делу
- **Зачем** — где это в реальной работе
- **Как спрашивают на собеседе** — реальные формулировки
- **Что проверяет интервьюер** — что за вопросом
- **Что сделать руками** — пошаговое задание
- **Как упаковать в резюме** — честная формулировка
- **Типичная ошибка** — где все спотыкаются



ГЛАВНЫЙ ПРИНЦИП ОТВЕТА НА СОБЕСЕ

Отвечай **просто** и не закапывай себя. «Что такое Kubernetes?» — «Оркестратор контейнеров. Есть **docker-compose** для одной машины; а чтобы запускать контейнеры на множестве машин, планировать их, следить за состоянием и мониторить — нужен Kubernetes». Сначала короткий понятный ответ, потом — глубже, если попросят. Без лишних англицизмов.



Не начинай с гигантского проекта. По каждой технологии — маленькое задание руками. Большой capstone — только в самом конце, когда базовые кирпичи уже собраны.



Базовый старт — первый сервер руками

С ЭТОГО НАЧИНАЕТСЯ ВСЁ ОСТАЛЬНОЕ

ПРОСТЫМИ СЛОВАМИ

Прежде чем грызть каждый инструмент по отдельности, собери из них **один простой работающий сервер**. Обычный бэкэнд + база в Docker, спереди nginx. Это скелет, на который потом ляжет всё — Docker, сети, CI/CD, Kubernetes.

— ЧТО СДЕЛАТЬ — ПОШАГОВО

- 1 **Арендуй самый дешёвый VPS** у любого провайдера. Зайди по `ssh`. Посмотри, что за машина:
`uname -a`, `df -h`, `free -m`.
- 2 **Поставь Docker и docker compose руками**. Не из готового образа — пройди установку сам, чтобы понять, что происходит.
- 3 **Подними простой бэкэнд + базу**. Любой HTTP-сервис и PostgreSQL — оба в `docker-compose.yml`. Спереди **nginx как gateway** (reverse-proxy на 80/443 → твой сервис).
- 4 **Проверь, что живое**. Открой в браузере, посмотри `docker ps`, логи `docker compose logs -f`, `curl localhost`. Где порты, кто кого слушает.
- 5 **Обнови код сервиса**. Пересобери образ, `docker compose up -d`. Посмотри, что происходит при обновлении: есть ли даунтайм, как откатиться на старый образ.
- 6 **Добавь вторую машину**. Вынеси базу на неё. Организуй **приватную сеть и связность** между машинами — сервис на одной ходит к базе на другой по приватному адресу.



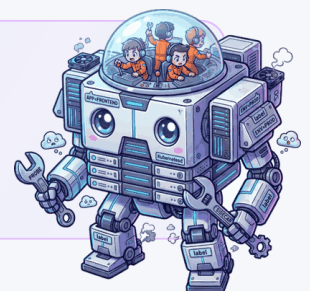
ЧТО ТЫ ПОЙМЁШЬ НА ЭТОМ ПРОЕКТЕ

Что такое порты и проксирование. Почему контейнер «эфемерен» и где живёт состояние. Как выглядит обновление и откат. И главное — **как машины общаются по сети**. Дальше ты будешь только углублять каждый из этих кусков.



КАК В РЕЗЮМЕ

«Развернул бэкэнд и PostgreSQL в Docker за nginx-gateway на арендованном сервере; вынес БД на отдельную машину с приватной сетевой связностью.»





Путь запроса — от браузера до базы

САМЫЙ ВАЖНЫЙ ПРАКТИЧЕСКИЙ БЛОК ДЛЯ СТАРТА

ЗАЧЕМ

Прежде чем лезть в Kubernetes — пойми, **как трафик доходит до сервиса**. Собери полный путь руками: домен → Nginx → backend → база по приватной сети. После этого DevOps перестаёт быть магией.

Браузер

HTTPS · порт 443

DNS: example.com → публичный IP сервера

Nginx — gateway

:80 / :443 · завершает TLS

proxy_pass внутрь

Backend

:8080 · наружу не торчит

приватная сеть

PostgreSQL

:5432 · только приватно

Снаружи открыт только **80/443**. Сертификат — на Nginx. Backend и база наружу не смотрят.

— 1 · DNS И ДОМЕН

Домен — человекочитаемое имя; DNS превращает его в IP. Сделай **A-record** на IP сервера, проверь:

`dig example.com`, `nslookup example.com`. Домена нет — временно пропиши в `/etc/hosts`.

— 2 · NGINX КАК GATEWAY

Nginx слушает 80/443, backend живёт на `127.0.0.1:8080`. Пользователь ходит **в Nginx**, тот проксирует внутрь. Прямой доступ к backend закрыт.

`/etc/nginx/sites-enabled/app.conf`

NGINX

```
server {
    listen 80; server_name example.com;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;           # какой домен спросил юзер
        proxy_set_header X-Real-IP $remote_addr; # реальный IP клиента
        proxy_set_header X-Forwarded-Proto $scheme; # http или https
    }
}
```

Проверь руками: `curl localhost:8080` (backend жив) и `curl -I http://example.com` (через Nginx). Логи — `journalctl -u nginx` и `tail -f /var/log/nginx/error.log`.



HTTPS и приватная сеть

— 3 • HTTPS И СЕРТИФИКАТ

Сертификат нужен, чтобы браузер доверял сайту и трафик был зашифрован. Без него — HTTP (порт 80), с ним — HTTPS (443). Сертификат выпускается **на домен**, не «на сервер».

- 1 Подними Nginx на 80, проверь, что домен открывается по HTTP.
- 2 Выпусти сертификат через **Let's Encrypt / Certbot**, включи HTTPS, сделай redirect HTTP → HTTPS.
- 3 Проверь: `curl -I https://example.com`, `openssl s_client -connect example.com:443`.
Автопродление: `certbot renew --dry-run`.



Нет реального домена — потренируйся на **self-signed** или `mkcert` локально (браузер не доверяет — это нормально, объясни почему).

— 4 • ПРИВАТНАЯ СЕТЬ МЕЖДУ МАШИНАМИ

Не всё должно торчать в интернет. **node1** — gateway/backend (открыты 80/443), **node2** — база, доступна **только по приватному IP node1**. База, слушающая `0.0.0.0` в интернет, — дыра.

- 1 Подними PostgreSQL на node2, закрой её от внешнего интернета (firewall / security group).
- 2 Разреши доступ к 5432 только с приватного IP node1. С node1 проверь: `ping`, `nc -vz <ip> 5432`, `psql -h <ip>`.
- 3 Снаружи убедись, что база недоступна. Диагностика: `ss -tulpn`, `ip a`, `ip route`, `ufw status`.



КАК В РЕЗЮМЕ

«Развернул сервис за Nginx-gateway с HTTPS (Let's Encrypt), вынес БД на отдельную машину, доступную только по приватной сети; настроил firewall и диагностику связности.»



ПРОСТЫМИ СЛОВАМИ

Linux — это операционка, на которой крутится почти весь прод. От тебя не ждут, что ты ядро напишешь. Ждут, что ты **зайдёшь на сервер и поймёшь, что на нём происходит**: что ест ресурсы, почему сервис упал, куда делось место.

— ЧТО ЗНАТЬ (РОВНО ТО, ЧТО СПРАШИВАЮТ)

- **Ресурсы**: `htop`/`iotop`, `load average` (почему бывает выше числа ядер при простаивающем CPU), `df` vs `du`
- **Диск и ФС**: LVM, `inode`'ы («место есть, запись падает»), структура папок — `/proc`, `/etc/fstab` и компания
- **Процессы и сигналы**: как остановить процесс; чем `SIGTERM` отличается от `SIGINT` и `SIGKILL`
- **Дескрипторы**: что такое FD; можно ли восстановить удалённый, но открытый процессом файл — из дескриптора
- **Загрузка и systemd**: что происходит при старте системы; как сделать системный сервис (`unit`-файл)
- **Изоляция**: `cgroups` и `namespaces` — обычно в контексте Docker; `user space` vs `kernel space`
- **Права и ссылки**: права доступа и как их выдавать; симлинки vs хардлинки
- **Перенос и сеть**: `rsync` vs `scp`; интерфейсы, открытые порты; `iptables`/`ipvs` (всплывает в `kube-proxy`)

— КАК СПРАШИВАЮТ НА СОБЕСЕ

- ? Чем `df` отличается от `du`? Покажет ли `df`, что место съел удалённый, но всё ещё открытый файл?
- ? `inode`'ы кончились, а место по `df -h` есть. Что произойдёт и как это диагностировать?
- ? Чем `SIGTERM` отличается от `SIGKILL` — и почему `kill -9` это не первый, а последний довод?
- ? Что происходит при загрузке системы? Что такое `systemd` и как сделать свой сервис?
- ? `cgroups` и `namespaces` — что это и при чём тут контейнеры?



ЧТО ПРОВЕРЯЕТ ИНТЕРВЬЮЕР

Работал ли ты с Linux **в проде руками** — или знаешь по книжке. Теоретик расскажет определение `inode`; практик видел сервер, который лёг из-за миллионов мелких файлов, и знает, что набрать первым.





Linux — что сделать руками

— ЗАДАНИЯ (СЛОМАЙ И ПОЧИНИ)

- 1 Подними node_exporter как systemd-сервис.** Напиши unit-файл, `systemctl enable --now`, собери метрики (`curl :9100/metrics`).
- 2 Забей inode'ы.** Создай тьму пустых файлов, поймай «No space left» при свободном `df -h`, найди виновника через `df -i`.
- 3 Поиграй сигналами.** Пошли процессу SIGTERM, SIGINT, SIGKILL — посмотри, кто ловится, кто нет.
- 4 Восстанови файл из дескриптора.** Удали открытый процессом файл, достань его из `/proc/<pid>/fd/`.
- 5 Права и ссылки.** `chmod` / `chown`, сделай симлинк и хардлинк — увидь разницу.

ШПАРГАЛКА • ЗАЙТИ И ПОНЯТЬ СЕРВЕР

<code>ss -tlnp</code>	открытые порты + кто слушает
<code>ip a • ip route</code>	интерфейсы • таблица маршрутов
<code>htop</code>	процессы, CPU, память, load avg
<code>df -h • df -i</code>	место по диску • по inode'ам
<code>du -sh *</code>	что занимает место в папке
<code>free -m</code>	оперативная память
<code>lsblk • vgs</code>	диски • тома LVM
<code>systemctl status</code>	состояние сервиса
<code>journalctl -u svc</code>	логи сервиса
<code>cat /etc/os-release</code>	какой дистрибутив
<code>docker ps • system df</code>	контейнеры • место docker
<code>lsof -p <pid></code>	открытые файлы процесса



КАК В РЕЗЮМЕ

«Диагностировал инциденты на Linux-серверах: исчерпание inode'ов, утечка дескрипторов, разбор по journald; поднимал сервисы как systemd-юниты с метриками node_exporter.»



ТИПИЧНАЯ ОШИБКА

Знать команды **теоретически**. На «покажи руками» — пауза. Linux — это мышечная память; она набивается только на живом сервере: сломай и почини.

ПРОСТЫМИ СЛОВАМИ

Сети — это про то, как данные доходят из точки А в точку Б, и что мешает им дойти. Модель OSI нужна **не для зубрёжки уровней**, а чтобы при сбое идти по слоям сверху вниз и локализовать обрыв: приложение → порт → маршрут → интерфейс.

— ЧТО ЗНАТЬ

- **Путь пакета:** что происходит при `curl example.com` — DNS → TCP-рукопожатие → TLS → HTTP. База диагностики.
- **Диагностика:** открытые порты, таблица маршрутов, интерфейсы, `ping/traceroute`, DNS.
- **Адреса:** приватный vs публичный IP, NAT, зачем VPN (site-to-site, напр. WireGuard).
- **Фильтрация:** firewall / iptables на пальцах; TLS-рукопожатие в двух словах.

— КАК СПРАШИВАЮТ

- ? Что происходит при `curl https://example.com` — по шагам?
- ? Как проверить, открыт ли порт на удалённом хосте?
- ? Где смотреть таблицу маршрутизации и что там искать?
- ? Зачем нужен TLS-хэндшейк и что в нём происходит?

— ЧТО СДЕЛАТЬ РУКАМИ • СЕТЬ МЕЖДУ ТРЕМЯ МАШИНАМИ

- 1 **Подними 3 виртуалки** (можно в облаке, в разных подсетях). Осмотрись: `ip a`, `ip route`, `ss -tlnp` — пойми каждую строчку вывода.
- 2 **Свяжи их приватно** — через WireGuard или peering. Проверь связность: `ping`, `traceroute` / `mtr`, `dig`.
- 3 **Сломай специально** — закрой порт файрволом или убери маршрут. Продиагностируй `tcpdump` на обоих концах: пакеты уходят? доходят? отбрасываются где?



ЛЮБИМЫЙ КЕЙС НА СОБЕСЕ • «ТРАФИК ДОХОДИТ, НО ОТВЕТ НЕ ВОЗВРАЩАЕТСЯ»

Классика асимметричной маршрутизации: запрос пришёл через один интерфейс, ответ ушёл через другой — и потерялся. Диагностика: `tcpdump` видит входящие, но не исходящие; `ip route get <client_ip>` — куда сервер шлёт ответ; проверить default route и firewall на обратном пути. **Это не угадывание — это движение по слоям.**



ТИПИЧНАЯ ОШИБКА

Учить сети теоретически и не трогать руками. «DNS — система доменных имён» знают все; а на «как проверить, куда резолвится домен» — пауза. `dig`, `ss`, `tcpdump`, `ip route` — это мышечная память.



ПРОСТЫМИ СЛОВАМИ

Контейнер — это изолированный процесс со своей файловой системой (не виртуалка — ядро общее). **Образ** — рецепт (слои + метаданные), **контейнер** — запущенный экземпляр образа. «У меня работает» уходит в прошлое: среда описана кодом и воспроизводима на ноуте, в CI и на сервере.

— ЧТО ЗНАТЬ

- **Слои и кэш:** каждая инструкция Dockerfile — слой; кэш инвалидируется сверху вниз.
- **Порядок «от общего к частному»:** сначала зависимости, потом код — иначе каждая правка сбрасывает установку зависимостей.
- **Multi-stage build:** стадия-сборщик + тонкий рантайм → маленький чистый образ.
- **Тег `:latest`** — зло в проде/CI: бери `<commit-sha>`. Состояние — в volume, контейнер эфемерен.
- **В основе — примитивы ядра Linux:** `namespaces` дают изоляцию (свой PID, сеть, точки монтирования, hostname), `cgroups` — лимиты ресурсов (CPU, RAM). Контейнер = процесс в своих namespaces под cgroups. Почитать: `man 7 namespaces`, `man 7 cgroups`.

```
Dockerfile — кэш + multi-stage DOCKER

# 1) сборка — тут компилятор и зависимости
FROM golang:1.22-alpine AS builder
WORKDIR /app

# сначала манифесты → слой кэшируется
COPY go.mod go.sum ./
RUN go mod download

# потом код — меняется чаще всего
COPY . .
RUN go build -o /server .

# 2) рантайм — только бинарь, без toolchain
FROM alpine:3.19
COPY --from=builder /server /server
ENTRYPOINT ["/server"]
```

— КАК СПРАШИВАЮТ

- ? Разница образа и контейнера?
- ? Что такое multi-stage и когда он нужен?
- ? Что такое слои и как работает кэш?
- ? Почему не `:latest` в проде?

— ЧТО СДЕЛАТЬ РУКАМИ

- 1 **Заверни обычный бэкэнд в Docker.** Собери, посмотри размер и слои: `docker history`, `docker images`.
- 2 **Поймай инвалидацию кэша:** поменяй строчку в коде — увидь, как зависимости ставятся заново. Переставь инструкции + добавь `.dockerignore`.
- 3 **Сделай multi-stage** и сравни размер до/после. Затей по sha, запусти в registry.

ТИПИЧНАЯ ОШИБКА • `COPY . .` В НАЧАЛЕ

Первое, что делают все. Выглядит логично — а на деле любая правка кода сбрасывает кэш установки зависимостей, и каждая сборка в CI тянется минутами вместо секунд. **Сначала копируй то, что меняется редко.**



ПРОСТЫМИ СЛОВАМИ

CI — на каждый пуш автоматом гоняются линтер, тесты, сборка (никакого «у меня локально работает»). CD — после проверок код сам едет на сервер. Всё это — пайплайн из стадий `lint → test → build → push → deploy`, описанный в одном файле `.gitlab-ci.yml`.

— ЧТО ЗНАТЬ

- **Структура:** stages, jobs, runners, артефакты, `rules` / `needs`.
- **Сборка в CI:** Kaniko (без docker-демона), тег по `$CI_COMMIT_SHORT_SHA`, push в registry.
- **Деплой через Ansible:** job по SSH запускает `ansible-playbook` — он и раскатывает сервис.
- **Секреты** — в маскированных переменных GitLab, не в репозитории. Стратегии: rolling / blue-green / canary.

```
.gitlab-ci.yml  CI / CD

stages: [build, deploy]
variables:
  IMAGE:
    $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

build:
  stage: build
  image: kaniko-project/executor:debug
  script:
    - /kaniko/executor --context .
      --destination $IMAGE

deploy:
  stage: deploy
  script:
    - ansible-playbook -i prod
      deploy.yml
      -e image=$IMAGE
  rules: [{ if:
    '$CI_COMMIT_BRANCH=="main"' }]

```

— ЧТО СДЕЛАТЬ РУКАМИ • ПАЙПЛАЙН, КОТОРЫЙ ДЕПЛОИТ MY-SERVICE

- 1 **Репозиторий** с простым сервисом и Dockerfile.
- 2 **build job** — Kaniko собирает образ, тег по sha, push в registry.
- 3 **deploy job** — по SSH запускает `ansible-playbook`, который тянет образ и перезапускает сервис.
- 4 **Проверь руками:** зайди на сервер — `docker ps`, `curl localhost`. Реально ли поднялось.



ТИПИЧНАЯ ОШИБКА • «ЗЕЛЁНЫЙ ПАЙПЛАЙН = ЗАДЕПЛОИЛОСЬ»

Нет. Пайплайн проверяет, что команда **запустилась** успешно — не что сервис **работает**. Playbook завершился кодом 0, флажок зелёный, а контейнер упал через 5 секунд из-за кривого env. **После деплоя — всегда явная проверка на сервере** (`docker ps`, healthcheck).





ПРОСТЫМИ СЛОВАМИ

Terraform создаёт инфраструктуру из кода — виртуалки, сети, диски в облаке. **Ansible** настраивает то, что уже создано — ставит пакеты, кладёт конфиги, поднимает сервисы. Короткая формула: Terraform про «что существует», Ansible про «что установлено и настроено».

— ЧТО ЗНАТЬ

- **Разделение ролей:** TF = provisioning, Ansible = configuration. Не пытайся делать всё одним.
- **Terraform:** декларативность, `plan` vs `apply`, `state` (нельзя терять и править руками), провайдеры.
- **Ansible:** инвентарь, плейбуки, роли, идемпотентность, agentless (по SSH), `serial:` для безопасной раскатки.

— КАК СПРАШИВАЮТ

- ? Разница Terraform и Ansible — когда что?
- ? Что такое state и почему его нельзя потерять?
- ? Что делает `plan` и что `apply`?
- ? Что такое идемпотентность в Ansible?
- ? Что будет, если поменять инфру руками мимо кода?

— ЧТО СДЕЛАТЬ РУКАМИ • МАЛЕНЬКИЙ СТЕНД, НЕ ПРОД

- 1 **Terraform поднимает 1–2 виртуалки + сеть** в облаке. `terraform init → plan → apply` — читай `plan` внимательно.
- 2 **Ansible настраивает их:** ставит docker, кладёт конфиг, поднимает простой сервис.
- 3 **Проверь идемпотентность:** прогони плейбук второй раз — должно быть `ok`, а не `changed`.
- 4 **Убери всё:** `terraform destroy`. Никаких ручных удалений в консоли.



ОШИБКА • DRIFT

Поправил что-то руками в консоли облака «на минутку» — забыл. `terraform plan` теперь хочет это снести. Правило: **всё через код**, никаких ручных правок.



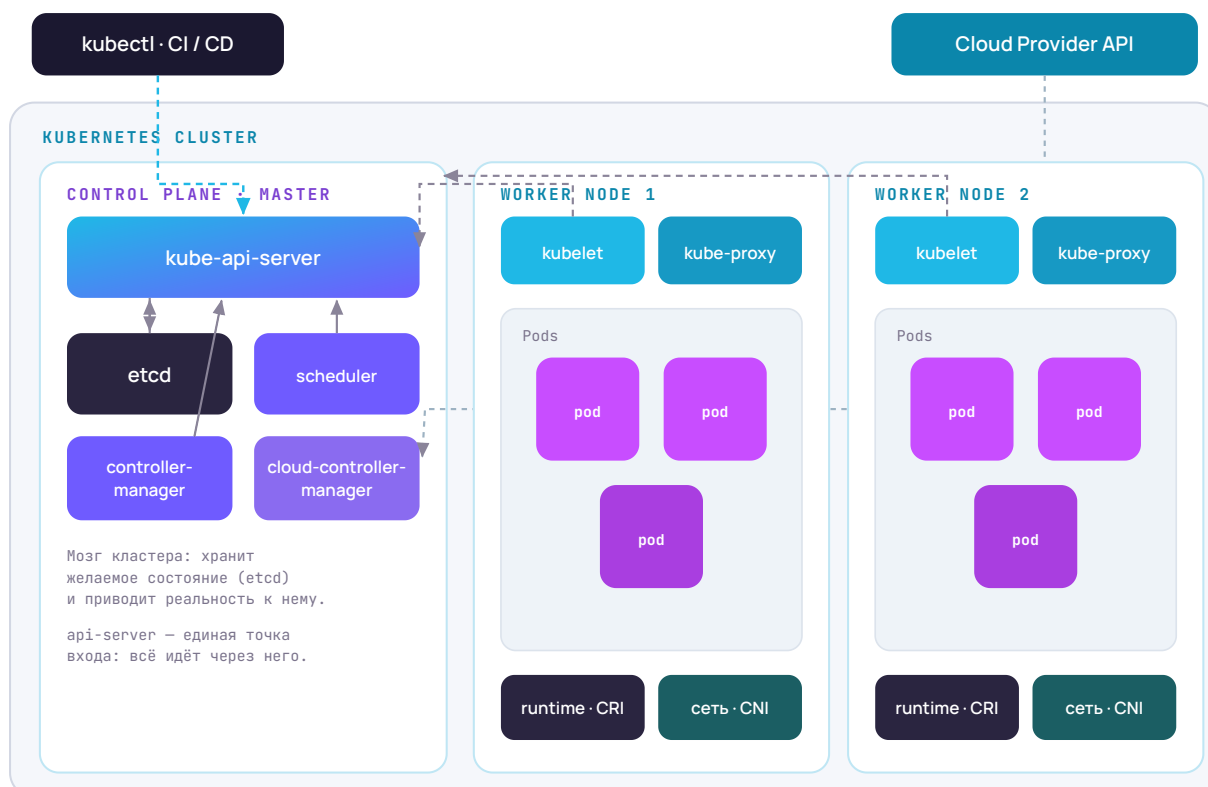
ОШИБКА • BLAST-RADIUS

Ansible без `serial` на 50 нод: одна ошибка (снёс default route) прокатилась по всем разом — потерял SSH ко всему. `serial: 1` ограничивает урон.



ЭТАЛОННЫЙ ОТВЕТ НА СОБЕСЕ

Оркестратор контейнеров. Есть `docker-compose` для запуска на одной машине; а чтобы запускать контейнеры **на множестве машин**, планировать их по узлам, следить за состоянием и автоматически восстанавливать — нужен Kubernetes. Дальше — глубже, если попросят.



Архитектура кластера. Control plane принимает решения; worker-узлы запускают pod'ы. Всё общение — только через **kube-api-server**.

- **api-server** — единая точка входа, через него идёт всё
- **etcd** — база, хранит желаемое состояние кластера
- **scheduler** — решает, на какой узел поставить pod
- **controller-manager** — следит, чтобы реальность сходилась с желаемым
- **cloud-controller-manager** — связь с облаком: LB, диски, узлы провайдера
- **kubelet** — агент на узле, запускает и держит pod'ы
- **kube-proxy** — сетевые правила, доступ к Service
- **CRI** — среда запуска контейнеров (containerd и т.п.)
- **CNI** — сеть подов (Calico / Cilium / Flannel): IP и связность
- **pod** — минимальная единица: один-несколько контейнеров вместе



Основные ресурсы и команды

ЧТО НУЖНО ЗНАТЬ • РЕСУРСЫ

Pod	минимальная единица запуска
Deployment	держит N реплик, rolling update + откат
Service	стабильный адрес + балансировка на pod'ы
Ingress	вход снаружи по http(s), маршрут по хостам
ConfigMap/Secret	конфиги и секреты отдельно от образа
PV / PVC	подключаемые тома для состояния
StatefulSet	stateful-приложения, стабильные имена
HPA	авто-масштабирование по метрикам
RBAC · NetworkPolicy	кто что может · сетевая изоляция

ШПАРГАЛКА • KUBECTL

<code>get pods -o wide</code>	pod'ы + узлы
<code>describe pod X</code>	события, причина падения
<code>logs -f X</code>	логи контейнера
<code>exec -it X -- sh</code>	зайти внутрь pod'a
<code>apply -f m.yaml</code>	применить манифест
<code>rollout undo deploy/X</code>	откатить деплой
<code>scale deploy/X --replicas=3</code>	сменить число реплик
<code>get events</code>	свежие события кластера
<code>top pod · top node</code>	потребление ресурсов

— КАК СПРАШИВАЮТ

- ? Чем Deployment отличается от ReplicaSet?
- ? Как один pod находит другой? (Service + DNS)
- ? scheduler смотрит на requests или limits?
- ? Зачем liveness/readiness probe?

— ЧТО СДЕЛАТЬ РУКАМИ

- 1 Локальный кластер (minikube/kind). Задеплой **Deployment + Service + Ingress**.
- 2 Сломай — поставь несуществующий образ, поймай `ImagePullBackOff`, разбери через `describe` / `logs`.
- 3 Накрути реплики, выкати плохую версию, `rollout undo`.



ТИПИЧНАЯ ОШИБКА • REQUESTS И LIMITS

scheduler выбирает узел по **requests** (что просишь), а **limits** — потолок в рантайме (CPU throttling, memory OOM-kill). Неадекватные значения → кластер «переполнен на бумаге» или pod'ы `OOMKilled`. И помни: **учить Kubernetes раньше сетей бессмысленно** — без них непонятно, почему pod не видит pod.

ПРОСТЫМИ СЛОВАМИ

Вместо десятка YAML-манифестов, которые надо применять руками и править под каждое окружение, ты описываешь один **шаблонизируемый чарт** с `values` — и ставишь, обновляешь, откатываешь приложение одной командой. Если Kubernetes — это ОС кластера, то Helm — это `apt` / `brew` поверх него.

— ЧТО ЗНАТЬ

- **Chart / values / release:** пакет, значения под окружение, конкретная установка в кластер.
- **Откат:** Helm хранит историю релизов → `helm rollback` возвращает рабочую версию одной командой.
- **Снести разом:** `helm uninstall` удаляет все ресурсы релиза сразу.
- `helm template` : отрендерить чарт в чистые манифесты локально — увидеть, что реально уедет в кластер.

```
helm — жизненный цикл HELM

# отрендерить локально — без кластера
$ helm template app ./chart -f values-prod.yaml

# поставить или обновить (идемпотентно)
$ helm upgrade --install app ./chart -f values-prod.yaml --atomic

# история и откат
$ helm history app
$ helm rollback app 1

# снести весь релиз разом
$ helm uninstall app
```

— ЧТО СДЕЛАТЬ РУКАМИ

- 1 Возьми манифесты из прошлого шага (Deployment + Service + Ingress), вынеси образ/реплики/хост в `values.yaml`, шаблонизируй `templates/`.
- 2 `helm template` → проверь рендер. `helm upgrade --install` → в кластер.
- 3 Выкати «плохую» версию, сделай `helm rollback`. Потом `helm uninstall`.

ТИПИЧНАЯ ОШИБКА • СЕКРЕТЫ В `VALUES.YAML`

Пароль к базе открытым текстом в git, потому что «репо внутренний». Шифруй чувствительные values (helm-secrets + SOPS) или храни в Vault/external-secrets. И ставь `--atomic` — иначе при ошибке в кластере останутся полу-применённые ресурсы.



ПРОСТЫМИ СЛОВАМИ

Мониторинг отвечает на два вопроса: «**работает ли?**» и «**почему сломалось?**». Три столпа: **метрики** — числа во времени (CPU, RPS, ошибки), **логи** — что именно случилось, **трейсы** — путь одного запроса через сервисы. Без этого прод — чёрный ящик.

— ЧТО ЗНАТЬ

- **Prometheus** — база данных временных рядов. Сам **ходит и собирает** (pull) метрики с `/metrics` сервисов, хранит как ряды, запросы — на **PromQL** (например `rate(http_requests_total[5m])`).
- **Grafana** — дашборды поверх источников. Сама данные **не хранит** — подтягивает из Prometheus / Loki и рисует графики.
- **Loki** — «Prometheus для логов»: дешёвое хранение по тем же лейблам. Собирают Promtail / Grafana Alloy. **VictoriaMetrics / VictoriaLogs** — экономнее по ресурсам замена Prometheus/Loki, тот же PromQL.
- **Виды метрик**: ресурсные (CPU/RAM/диск), кластерные (поды, рестарты), **бизнес** (заказы, регистрации) и golden signals — latency, traffic, errors, saturation.
- **Alertmanager** — правила на PromQL → алерт в Telegram/почту, когда «плохо».

```
kube-prometheus-stack —
весь стек одним чартом  BASH

# 1) ставим
Prometheus+Grafana+Alertmanager разом
helm repo add prometheus-community \
https://prometheus-
community.github.io/helm-charts
helm install kps \
prometheus-community/kube-
prometheus-stack

# 2) внутри уже есть node-exporter +
дашборды k8s
kubectl port-forward svc/kps-grafana
3000:80

# 3) открыть в браузере — готовые
графики
http://localhost:3000 # admin / prom-
operator
```

— КАК СПРАШИВАЮТ

- ? Чем метрика отличается от лога и трейса?
- ? Pull или push? Как Prometheus собирает метрики?
- ? Что такое `rate()` и зачем PromQL?
- ? Grafana хранит данные? Откуда берёт?

— ЧТО СДЕЛАТЬ РУКАМИ

- 1 Поставь **kube-prometheus-stack** в **minikube** и открой Grafana — посмотри готовый дашборд по нодам и подам.
- 2 Отдай `/metrics` из своего сервиса (client-библиотека) и построй график `rate(http_requests_total[5m])`.
- 3 Настрой алерт «pod рестартует» в Alertmanager → уведомление в Telegram.



ПОТОК ДАННЫХ · ЗАПОМНИ СХЕМУ

Сервис отдаёт `/metrics` → **Prometheus** собирает и хранит → **Grafana** рисует дашборды → **Alertmanager** шлёт алерт. Логи: сервис → **Loki** → та же Grafana. Это «соберись с нуля» проект уровня собеса.



ГЛАВНОЕ

Назвать «Prometheus + Grafana» — мало. Спрашивают, **чем собирал и КАК настраивал**, и гоняют по деталям. Разберись с тремя вещами ниже — и сойдёшь за человека, который реально админил метрики.

— ЧТО ЗНАТЬ

- **3 вида метрик:** инфраструктурные (CPU/RAM/сеть/диск + **IOPS** — про него забывают), прикладные (статусы ответов, up/down, метрики рантайма) и функциональные/бизнес (уникальны под приложение).
- **Метрика = временной ряд:** имя + значение во времени + **лейблы**. По лейблам фильтруешь: сервис, namespace, приложение.
- **Cardinality** — не клади бизнес-ID в лейбл. Лейбл `user_id` → временных рядов столько же, сколько юзеров → **Prometheus умирает**. Гранулы — крупные.
- **RED + USE:** RED (Rate, Errors, Duration) — для прикладных; USE (Utilization, Saturation, Errors) — для инфраструктурных.

— LATENCY — ЭТО НЕ RESPONSE TIME

- **Latency** — backend замеряет сам себя: получил запрос → отдал ответ.
- **Response time** — полное время до пользователя. Поэтому **response time всегда больше latency** (его часто и не замерить).



Чаще спрашивают про **latency** — response time замерить сложно.



KILLER-ВОПРОС • НЕЛЬЗЯ АЛЕРТИТЬСЯ ПО СРЕДНЕМУ

10 запросов по 1 мс и один на 10 с → **среднее меньше секунды**, а проблема как раз в 10-секундном. Смотри **перцентили**: 90% лучших и 10% худших — отдельно (p90 / p99). Это касается всего мониторинга.

— КАК СПРАШИВАЮТ (РЕАЛЬНЫЙ РАЗНОС)

- ? Чем latency отличается от response time?
- ? Почему нельзя алертиться по среднему?
- ? Чем собирал и **как настраивал**?
- ? Какой компонент VictoriaMetrics за алерты? (`vmalert`) Куда AlertManager?



СВЯЗКА

Три столпа работают вместе: по метрике увидел всплеск ошибок → по трейсу нашёл, на каком сервисе → по логам понял причину. Ключ ко всему — **сквозной ID запроса**.

— ТРЕЙСЫ

- **Jaeger + OpenTracing** — стандартный ответ. **Trace ID** (= request ID) — сквозной на весь запрос; **Span ID** — один шаг (сервис → сервис → база).
- **Гоча: trace ID теряется** в Postgres / Kafka, если не проставить руками → разрыв цепочки → трассировка не работает.
- **В логах обязательно trace ID + span ID**. Нашёл проблемный trace → фильтр по нему в Kibana — и сразу нужные строки.

— ЛОГИ

- Приложение пишет в **stdout/stderr** → агент (**Fluent Bit**) собирает → хранилище. На себе говори просто «Fluent».
- **ELK** = Elasticsearch + Logstash + Kibana. Elastic стал платным → берут форк **OpenSearch** (полнотекстовый поиск). **Loki** — дешевле, через Grafana, но поиск слабее.
- **Индексы + ротация** — что спрашивают: один индекс на приложение (чтобы видеть сквозные логи), ротация ~7 дней (иначе кончится диск).

⚡ **Путь лога:** приложение (**stdout/stderr**) → агент собирает и обогащает (namespace, pod, labels) → хранилище → поиск/дашборд.

— АГЕНТЫ СБОРА (НА КАЖДОЙ НОДЕ)

- **Fluent Bit** — лёгкий (C), стандарт в k8s как DaemonSet. **Kubernetes-фильтр** обогащает логи метаданными pod / namespace / labels (без него на себе валятся).
- **Fluentd** — старший брат (Ruby), больше плагинов, тяжелее. **Promtail / Grafana Alloy** — агенты под Loki. **Vector / Logstash** — мощная обработка и трансформация на лету.

— ГДЕ ХРАНЯТ И ИЩУТ

- **ELK / OpenSearch + Kibana** — полнотекстовый поиск, богатые дашборды. Дорого по ресурсам: hear, диск, число шардов, ILM-ротация.
- **Loki + Grafana** — дешево, индексирует только лейблы. **Graylog** — готовый продукт с UI. **ClickHouse** — для огромных объёмов, читает прямо из Kafka.

КУДА ЭТО РАСТЁТ • СОВРЕМЕННЫЙ СТЕК

OpenTelemetry — единый стандарт телеметрии + **Collector** (универсальный адаптер, «как переходник для розеток»). При нагрузке — **Kafka**-буфер; хранят в **ClickHouse** (колоночная, сама читает Kafka). Stateful (Prometheus / OpenSearch) — на отдельных VM, не в кубе. Дашборд Grafana — это **JSON, то есть код**.



КАК УСТРОЕНО В K8S

PV — ресурс (диск), **PVC** — запрос приложения «дай мне N ГБ», **StorageClass** — динамический провижининг через **CSI**-драйвер. Под капотом — облачный диск или on-prem кластер хранилища.

— РЕЖИМЫ ДОСТУПА — СПРАШИВАЮТ!

- **RWO** (ReadWriteOnce) — том на ОДНУ ноду. Блочные диски (EBS, Ceph RBD, Longhorn). Для БД — то, что надо.
- **RWX** (ReadWriteMany) — на МНОГО нод сразу. Только файловые: **CephFS**, **NFS**, **SeaweedFS**. Для общих файлов, не для БД.
- **Object (S3 / MinIO)** — не PV вовсе: артефакты, бэкапы, кэш. Лучше «дисков в поде».

— ЧТО ВЫБРАТЬ

- **Облако**: CSI провайдера (EBS / GCE PD) — просто и надёжно.
- **On-prem, попроще**: **Longhorn** (RWO, лёгкий, UI) или NFS-provisioner.
- **On-prem, серьёзно**: **Ceph** (Rook) — block + file + object разом, но сложный. **SeaweedFS** — легче, с RWX.

— RWX (ОБЩИЙ ДОСТУП НА МНОГО НОД) — КТО ПОДДЕРЖИВАЕТ

NFS — RWX

CephFS — RWX

SeaweedFS — RWX

Longhorn — RWO (RWX через NFS)

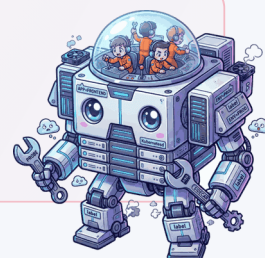
Ceph RBD — RWO

EBS / CSI block — RWO



ГОЧА • ГДЕ ТЕРЯЮТ ДАННЫЕ

NFS плохо для БД (блокировки, latency). **Локальный PV**: упала нода → данные недоступны, под не переедет. Для важного — реплицируемое хранилище (Ceph / Longhorn в 3 реплики) + бэкап (snapshot / Velero).



ЧТО ЭТО

Гипервизор делит одно железо на несколько изолированных **ВМ** — у каждой своя ОС и ядро (в отличие от контейнера — там ядро общее). **Зачем:** утилизация железа, изоляция, стенды, снапшоты, живая миграция.

— ЧТО ЗНАТЬ

- **Type 1** (на железе): **VMware ESXi**, **Proxmox/KVM**, **Hyper-V** — для прода и серверов.
- **Type 2** (поверх ОС): **VirtualBox**, **VMware Workstation** — для локальных стендов.
- **Proxmox VE** — open-source (KVM + LXC): кластер нод, storage (local-lvm / ZFS / Ceph / NFS / iSCSI), шаблоны cloud-init, есть Terraform-провайдер.
- **VMware vSphere** — enterprise (платный): vCenter, vMotion (живая миграция), DRS (балансировка).

— КАК ЭТО В DEVOPS

- **IaC:** Terraform-провайдер создаёт ВМ (диски, сеть, cloud-init), дальше **Ansible** настраивает.
- **Типовой стенд:** 3 ВМ → Kubespray → k8s-кластер. Снапшот перед экспериментом → сломал → откатил.
- **Storage backend важен:** ZFS (снапшоты), Ceph (HA), NFS (общий), local-lvm (просто).



КАК ПОПРАКТИКОВАТЬСЯ

Домашнее: **Proxmox** на старом ПК (или вложенная виртуализация), либо **VirtualBox / multipass** для пары ВМ. Подними **3 ВМ** → поставь кластер Kubespray-ем → сломай и почини. Это ровно то, что показываешь на собеседовании.





Что спрашивают на реальных собеседах

Формат каждого вопроса: **плохой ответ** → **нормальный короткий ответ** → **что проверяет интервьюер** → **как усилить**. Главное правило — отвечай просто, не закапывай себя терминами.

01

Что такое Kubernetes? (любимый вводный)

■ ПЛОХОЙ ОТВЕТ

«Это система для управления контейнерами» — заученное определение, которое ничего не объясняет.

■ НОРМАЛЬНО

Оркестратор контейнеров. Есть `docker-compose` для одной машины; а чтобы запускать контейнеры на множестве машин, планировать их по узлам, следить за состоянием и восстанавливать упавшие pod'ы — нужен Kubernetes.

■ ЧТО ПРОВЕРЯЕТ

Понимаешь ли, зачем вообще нужен Kubernetes, — или просто выучил определение.

■ КАК УСИЛИТЬ

Добавь: Deployment, Service, scheduler, желаемое состояние (etcd), health checks, rollout/rollback. Но только если попросят глубже.

02

Чем Deployment отличается от ReplicaSet? (частый)

■ ПЛОХОЙ ОТВЕТ

«Deployment — это когда много pod'ов» — неверно, сразу выдаёт новичка.

■ НОРМАЛЬНО

ReplicaSet держит нужное число pod'ов. Deployment управляет ReplicaSet'ами — даёт rolling update и откат через историю версий.

■ КАК УСИЛИТЬ

`kubectl rollout undo` переключает на предыдущий RS. Стратегии: RollingUpdate (по умолчанию) и Recreate (даунтайм, но без двух версий разом).

03

Чем контейнер отличается от виртуальной машины?

■ ПЛОХОЙ ОТВЕТ

«Контейнер — это лёгкая виртуальная машина» — неверно, сразу выдаёт поверхностное понимание.

■ НОРМАЛЬНО

Контейнеры используют **ядро хостовой ОС** и изоляцию через namespaces/cgroups, а VM запускает **отдельную гостевую ОС** поверх гипервизора. Поэтому контейнер легче и стартует за секунды.

■ КАК УСИЛИТЬ

Плотность размещения на хосте выше, образ воспроизводим. Проверяют: понимаешь ли изоляцию, а не зубришь определение.



Разбор вопросов — **сигнальные ловушки**

04 inode'ы кончились, место есть. Что произойдёт? (сигнальный)

- **ПЛОХОЙ ОТВЕТ** «Диск переполнен, надо почистить место» — но `df -h` показывает свободное место, человек в тупике.
- **НОРМАЛЬНО** Запись новых файлов упадёт с «No space left on device», хотя место есть. Кончились **inode'ы**. Проверить: `df -i`.
- **ЧТО ПРОВЕРЯЕТ** Реально ли ты работал с Linux в проде — практик видел сервер с миллионами мелких файлов.

05 Что происходит при `curl example.com` — по шагам?

- **НОРМАЛЬНО** DNS-резолвинг → TCP-рукопожатие (SYN/SYN-ACK/ACK) → TLS → HTTP GET → ответ.
- **КАК УСИЛИТЬ** До DNS: кэш → `/etc/hosts` → `/etc/resolv.conf` → рекурсивный resolver → корневые NS → TLD → authoritative NS → A-запись.
- **ЧТО ПРОВЕРЯЕТ** Умеешь ли «видеть» путь пакета по слоям и сказать, на каком именно уровне сломалось.

06 Rolling-деплой убивает pod в середине обработки. Как исправить?

- **НОРМАЛЬНО** Три части: `terminationGracePeriodSeconds` (время на завершение) + `preStop hook` (перестать брать задачи) + приложение ловит SIGTERM и доделывает текущее.
- **КАК УСИЛИТЬ** Для файлов — очередь с подтверждением: файл не удаляется, пока обработка не подтверждена. Это production-паттерн.

📄 Дальше — три самых «боевых» вопроса, на которых валятся: **502**, **CrashLoopBackOff**, **OOMKilled**.



Сломалось в проде — что делаешь?

Это вопросы «на руки»: интервьюер хочет услышать ход диагностики, а не определение. Отвечай как при реальном инциденте — по шагам.

07 Сайт отдаёт 502 Bad Gateway . Что делаешь первым?

- **ПЛОХОЙ ОТВЕТ** «Перезапусти nginx» — пальцем в небо, без диагностики.
- **НОРМАЛЬНО** 502 = прокси (nginx) не дождался ответа от бэкенда. Иду по слоям: жив ли бэкенд (`kubectl get pods / docker ps`), отвечает ли напрямую (`curl` в обход прокси), логи nginx, таймауты, upstream-адрес.
- **ЧТО ПРОВЕРЯЕТ** Идёшь ли ты системно от симптома к причине или тыкаешь наугад.

08 Pod в CrashLoopBackOff . Как чинить?

- **ПЛОХОЙ ОТВЕТ** «Удалю под, пусть пересоздастся» — он так же упадёт, причина не найдена.
- **НОРМАЛЬНО** Контейнер падает сразу после старта, k8s перезапускает по нарастающей паузе. Смотрю: `kubectl logs --previous` (логи упавшего), `kubectl describe pod` (события, exit code). Частое: кривой конфиг/env, не та команда, провал readiness, нет доступа к зависимости.
- **КАК УСИЛИТЬ** Exit 0 vs ненулевой; OOMKilled в describe; отличай liveness-перезапуск от падения процесса.

09 Pod OOMKilled . Почему и что делать? (requests vs limits)

- **ПЛОХОЙ ОТВЕТ** «Памяти не хватило на ноде» — не обязательно; чаще pod упёрся в свой `limit`.
- **НОРМАЛЬНО** Контейнер превысил **memory limit** — ядро его убило (OOM). scheduler ставит pod по **requests** (что просишь), а **limit** — потолок в рантайме. Поднять limit или починить утечку.
- **ЧТО ПРОВЕРЯЕТ** Понимаешь ли разницу requests vs limits и QoS (Guaranteed / Burstable / BestEffort).



ОБЩИЙ ПРИЁМ

На любой «сломалось» отвечай по структуре: **симптом** → **гипотеза** → **чем проверю** → **следующий слой**. Интервьюер слышит не факты, а то, как ты думаешь под инцидентом.

6 ошибок, которые стоят людям оффера

01 Начинать с Kubernetes

ОПАСНО Без сетей непонятно, почему pod не видит pod. Без Linux — почему OOMKilled.

КАК НАДО **Порядок:** Linux → сети → troubleshooting → Docker → CI/CD → IaC → K8s.

02 «Apply = применилось»

ОПАСНО Pipeline мог прогнать только plan. Ansible завершился без ошибки, но правило не раскатилось.

КАК НАДО **Верифицируй результат напрямую:** `kubectl get, systemctl status, iptables -L`.

03 Игнорировать blast-radius

ОПАСНО Реальный инцидент: изменён default route на всех хостах через Ansible — потеря SSH ко всем нодам.

КАК НАДО **serial: 1** для сетевых изменений. Terraform plan — смотреть на destroy.

04 Хранить секреты в git

ОПАСНО Даже если удалишь — остаётся в истории. Боты мониторят GitHub/GitLab на ключи в реальном времени.

КАК НАДО **HashiCorp Vault** + JWT-auth в CI. `.env` в `.gitignore`.

05 Резюме «участвовал в...»

ОПАСНО HR видит сотни резюме. «Участвовал в развёртывании K8s» — нет конкретики, нет доверия.

КАК НАДО **Формула:** контекст → действие → результат. «Развернул HA K8s-кластер... время деплоя с X до Y».

06 Готовиться только по видео и статьям

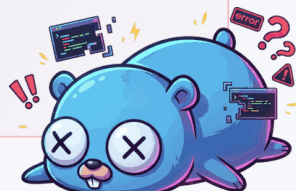
ОПАСНО На собеседе дадут сломанный сервис и попросят чинить вслух, в реальном времени. По конспектам это не тренируется.

КАК НАДО **Подними стенд → сломай специально → почини → объясни.** Тренируйся только на сломанном, а не на работающем.



ГЛАВНАЯ ОШИБКА

Ждать «идеальной готовности». Её не существует. Двигайся итерациями — собес, фидбэк, доработка.



От minikube к боевому кластеру

Не лезь сразу в продакшен-кластер. Сначала пощупай Kubernetes **локально на minikube**, освойся с `kubectl` и манифестами — а потом разворачивай боевой кластер по-взрослому через kubespray и доводи до полноценной платформы.

1

minikube — локально, без боли

Поставь **minikube** и **kubectl** с официальных сайтов (minikube.sigs.k8s.io, kubernetes.io).

`minikube start --driver=docker` — и через пару минут у тебя кластер. Тренируйся: `Deployment` + `Service` + `Ingress`, `kubectl get/describe/logs`, сломай `ImagePullBackOff`, накрути реплики, `rollout undo`.

2

kubespray — кластер на 3 ноды «в бою»

Когда освоился — разверни **настоящий кластер на 3 нодах** через kubespray: распределённый control-plane, etcd, cri-o. Это уже как на реальной проверке (следующая страница — пошаговая лаба).

3

финальный проект — доставка и наблюдаемость

Задеплой в кластер **фронтенд + бэкенд из GitLab**, добавь метрики (kube-prometheus-stack) и логи (Loki в Grafana). Полноценная платформа — то, что показывают на собесе.



С ЧЕГО НАЧАТЬ ПРЯМО СЕЙЧАС

```
minikube start → kubectl create deploy web --image=nginx
```

```
→ kubectl expose deploy web --port=80 →
```

```
minikube service web
```

Пять минут — и ты уже гоняешь приложение в Kubernetes. Дальше только усложняешь.



Кластер на kubespray — пошаговое задание

Формат — как на настоящей лабе: конкретные шаги, конкретные команды, проверяемый результат. Смысл — почувствовать, что control-plane распределённый: гасишь компоненты на одной ноде, а кластер живёт через другую.

Развернуть и «потрясти» кластер

ЛАБА

kubespray • 3 ноды • etcd • cri-o

ШАГИ

1. Разверни кластер на 3 нодах последней стабильной версией **kubespray** : control-plane на **node1** и **node2**; **etcd** на **node1**, **node2**, **node3** (тип запуска host); среда запуска — **cri-o** .
2. Сохрани бэкап etcd на node1 в **/tmp/node1.etcd.backup** . Для подключения укажи сертификаты (иначе **etcdctl** зависнет), добавь флаг **--debug** , чтобы видеть ошибки.
3. Останови kube-apiserver на node2: убери манифест **/etc/kubernetes/manifests/kube-apiserver.yaml** из папки и **systemctl restart kubelet** .
4. Перенастрой kubectl на node2 на ip node1 — поменяй адрес api-сервера в **kubeconfig** .
5. Останови etcd на node2: **systemctl stop** .
6. Запусти под **alpine** с командой **ping 8.8.8.8** , используя kubectl с node2.
7. Убедись, что кластер жив через node1 и под работает — задание сдано.

КРИТЕРИИ

- ✓ бэкап etcd лежит в **/tmp/node1.etcd.backup**
- ✓ после остановки apiserver+etcd на node2 кластер отвечает через node1
- ✓ под alpine с node2 запущен и пингует **8.8.8.8**



Зачем: ты руками убедился, что несколько control-plane и кворум etcd — это не теория. Убил apiserver и etcd на одной ноде — кластер продолжил работать через другую. Вот это и рассказывай на собесе.



Доставка из GitLab и наблюдаемость

На рабочем кластере собери полноценную платформу: приложение едет из GitLab само, а ты видишь его метрики и логи в одном окне. Это уже разговор уровня middle.

- 1 Приложение — фронтенд + бэкенд.** Образы собираются в GitLab CI (Kaniko, тег по `$CI_COMMIT_SHORT_SHA`), пушатся в registry.
- 2 Доставка в кластер.** Заведи `ServiceAccount` + RBAC, отдай kubeconfig в GitLab. Деплой из пайплайна через **Helm** (`upgrade --install`, при сбое — `rollback`). Снаружи — Ingress + TLS через cert-manager.
- 3 Метрики — kube-prometheus-stack.** Поставь Helm-чартом: Prometheus + Grafana + Alertmanager одним релизом. Открой Grafana — там уже готовые дашборды по кластеру, нодам и подам.
- 4 Логи — Loki + Grafana.** Поставь Loki и сборщик (promtail / Grafana Agent). Теперь логи всех подов — в той же Grafana, рядом с метриками. Настрой пару алертов в Alertmanager.

GitLab CI

Helm

ServiceAccount + RBAC

cert-manager

Prometheus + Grafana

Alertmanager

Loki



КАК В РЕЗЮМЕ

«Собрал платформу в Kubernetes: доставка фронтенда и бэкенда из GitLab CI через Helm (service account, rollback); наблюдаемость — kube-prometheus-stack (Prometheus + Grafana + Alertmanager) и централизованные логи через Loki.»



Отказоустойчивость и выбор кластера

Восьминодовый HA-кластер строить руками для портфолио — оверкилл. Но **понимать, как устроена отказоустойчивость**, нужно: это любимая тема на собеседе. Разберись в принципе, а не в зубрёжке.

KEEPALIVED + HAProxy — ЗАЧЕМ

Один control-plane — единая точка отказа: упал master, кластером не поуправлять. Решение: **несколько control-plane за балансировщиком HAProxy**, а **keepalived** даёт «плавающий» виртуальный IP (VIP), который автоматически переезжает на живой балансировщик. Клиенты (`kubectl`, узлы) всегда стучатся в один VIP — и не замечают, что под ним сменился сервер. Это и есть базовый кирпич отказоустойчивости; рядом — обязательный кворум `etcd` (нечётное число узлов).

Облачный кластер или свой — как отвечать

	MANAGED (В ОБЛАКЕ)	SELF-HOSTED (СВОЙ)
Control-plane	держит провайдер, ты не думаешь	поднимаешь и обслуживаешь сам (etcd, апгрейды)
Сложность	низкая — быстро в прод	высокая — полный контроль
Стоимость	платишь за узлы (иногда и за control-plane)	железо/VPS без наценки за managed
Когда брать	мало рук, нужно быстро и надёжно	требования к данным/локации, экономия, обучение



КАК ОТВЕЧАТЬ НА СОБЕСЕ

На вопрос «managed или свой» нет единственно верного ответа — есть **trade-off**. Назови оба варианта, плюсы-минусы и условие выбора. Именно это и хочет услышать интервьюер: что ты думаешь о компромиссах, а не зубришь «правильный» ответ.



Резюме: **слабо** → **сильно**

Рекрутер тратит 5–7 секунд. HR видит сотни резюме. Формула опыта: **Контекст** → **Действие** → **Результат** → **Технологии**.

СЛАБО → СИЛЬНО

8 формулировок из реальных DevOps-резюме

СЛАБО

СИЛЬНО

«Работал с Kubernetes»

«Развернул HA K8s-кластер (8 нод, kubeadm + Ansible), настроил etcd-кворум, HAProxy+keepalived перед control-plane»

«Настраивал CI/CD»

«Унифицировал GitLab CI для 80+ микросервисов через шаблоны — время подготовки pipeline с дней до < 1 часа»

«Работал с мониторингом»

«Настроил Prometheus + Grafana, добавил алертинг — MTTR снизился с 2 ч до 10–15 мин»

«Знаю Terraform»

«IaC на Terraform + Ansible: поднятие нового сервера сократилось с 3–4 ч до ~10–15 мин»

«Знаю Docker»

«Kaniko-сборка (rootless, без docker daemon). Registry-mirror — устранил падения из-за таймаутов docker.io»

«Участвовал в инцидентах»

«Диагностировал 504 timeout (ровно 60 с) на scoring-API — корень в дефолтном proxy_read_timeout nginx. Фикс через CI»

«Опыт с Vault»

«Внедрил HashiCorp Vault для 80+ микросервисов, интегрировал с GitLab CI через JWT-auth»

«Работал с VPN»

«Site-to-Site VPN (WireGuard) между Hetzner и Yandex Cloud — приватная связность без публичных endpoint'ов»



ОШИБКА НОВИЧКА

Писать «коммуникабельный, стрессоустойчивый». Парсеры ищут точные слова. **«Postgre» вместо «PostgreSQL»** — вылетаешь из фильтра.





Готов ли ты к DevOps-собеседованию

Честно отметить, что уже умеешь. Незакрытые пункты — это твой **план повторения** перед собесом. Эту страницу имеет смысл сохранить отдельно.

4 / 10



Уверенно работаю в Linux-терминале



Настроил CI/CD-пайплайн от пуша до деплоя



Понимаю сети: DNS, TCP, порты, маршрутизацию, TLS



Понимаю базовые сущности Kubernetes



Умею диагностировать проблему по слоям



Могу объяснить свои проекты в резюме



Понимаю Docker и docker-compose



Прошёл mock-интервью или проговорил ответы вслух



Собрал хотя бы один проект руками



Знаю слабые места и что повторять перед собесом



КАК ЭТИМ ПОЛЬЗОВАТЬСЯ

Не нужно закрыть все 10 пунктов перед первым откликом. Закрой первые 5–6 — и иди на разведочные собесы: они сами покажут, что повторить. **Готовность набирается на собеседах, а не до них.**

Выбери свой старт — с какой точки ты заходишь

У всех разная база. Сисадмину не надо начинать как разработчику, а студенту нельзя прыгать сразу в Kubernetes. Выбери маршрут ближе к своей точке.

СИСАДМИН Уже есть Linux, серверы, сети, bash

4–6 мес

добавить: Docker GitLab CI Terraform / Ansible Kubernetes Helm

козырь Понимаешь инфраструктуру — этого нет у пришедших из кода. слабое «Настроил руками и забыл». DevOps требует повторяемости: IaC, pipeline, monitoring.

▶ **Первый шаг:** подними сервис через Docker Compose на VPS, потом переведи ручную настройку сервера в Ansible.

РАЗРАБОТЧИК Есть код, Git, понимание приложения

3–5 мес

добавить: Linux глубже сети Docker CI/CD как владелец Kubernetes monitoring

козырь Понимаешь, что именно деплоится — легче проектировать pipeline и чинить. слабое Проседают Linux, сети, DNS, TLS, firewall, systemd — на себе быстро вскрывается.

▶ **Первый шаг:** задеплой свой backend на VPS через Nginx + Docker Compose + GitLab CI.

СТУДЕНТ / СТАРТ Базы мало — идём по порядку

6–10 мес

добавить: Linux сети Git Docker CI/CD IaC Kubernetes

козырь Не переучиваешься — сразу строишь правильную систему знаний. слабое Нельзя прыгать в Kubernetes без базы — будет «проходил, но не объясню».

▶ **Первый шаг:** SSH, базовые Linux-команды, nginx, простой backend на VPS и диагностика сети.





Полезные ссылки — учись по лучшему

Проверенные бесплатные ресурсы по каждой теме. Ссылки кликабельны прямо из PD
сохрани эту страницу как карту обучения.

СТАРТ • ROADMAP

- **DevOps Roadmap** roadmap.sh/devops
- **DevOps Exercises** github.com/bregman-arie

LINUX

- **Linux Journey** linuxjourney.com
- **Шпаргалки-комиксы** wizardzines.com
- **OverTheWire: Bandit** overthewire.org

- **SadServers — чини сломанные серверы вживую** sadservers.com • лучший тренажёр к собесу

СЕТИ

- **Beej's Guide to Networking** beej.us
- **High Performance Browser Networking** hpbn.co

DOCKER

- **Официальная документация** docs.docker.com
- **Docker Curriculum** docker-curriculum.com
- **Play with Docker** labs.play-with-docker.com

CI / CD

- **GitLab CI/CD docs** docs.gitlab.com/ci

IAC • TERRAFORM / ANSIBLE

- **HashiCorp Learn (Terraform)** developer.hashicorp.com
- **Ansible docs** docs.ansible.com

KUBERNETES

- **Официальная документация** kubernetes.io/docs
- **Kubernetes the Hard Way** github.com/kelseyhightower
- **Killercoda — интерактив** killercoda.com
- **minikube** minikube.sigs.k8s.io

HELM • НАБЛЮДАЕМОСТЬ

- **Helm docs** helm.sh
- **Prometheus docs** prometheus.io
- **Grafana tutorials** grafana.com/tutorials



А САМЫЙ МОЩНЫЙ РЕСУРС — ЭТО СИСТЕМА

Ссылки выше дадут базу, чтобы «копнуть вглубь». Но **быстрее всего** путь проходят там, где есть реальные стенды, обратная связь и банк вопросов — на **BoostMentor**: 1165+ реальных вопросов, mock-интервью, карьерный трек и ментор **Виктор Шутов**.

[@B000STMENTOR](https://t.me/B000STMENTOR) • boostmentor.ru

• АВТОР • ПРАКТИК, А НЕ ТЕОРЕТИК

Виктор Шутов — за гайдом реальный опыт

Не инфобизнес. Реальный путь через рынок, собеседования, офферы и обучение людей — и собственная платформа за плечами.



Виктор Шутов

@B000STMENTOR
boostmentor.ru



10 лет опыта в IT

В IT с 2016:
разработчик →
DevOps / SRE.



**Оффер Senior
DevOps за 4 дня**

И без опыта в DevOps
— с нуля до Senior.



**Рекорд без
премий — 450
000 ₽**

На руки, без бонусов.



**С премиями —
567 000 ₽ на руки**

Оклад 420 000 + 35%
ежемесячная премия.



**Сотни реальных
собеседований**

DevOps, SRE, Backend
Go, Backend C#.



**Больше 50
техсобесов**

Выложил на
платформе для
студентов.



**Курс «Go на
миллион»**

3 человека на доход 1
млн; ещё несколько
удвоили.



**Основатель
boostmentor.ru**

AI-платформа.
Преподаю с 2012,
МАИ.



КАК ПРОЙТИ ЭТОТ ПУТЬ БЫСТРЕЕ

Эти цифры — результат системной подготовки. Тот же путь быстрее всего проходят с **BoostMentor**: реальные стенды, банк вопросов, моск-интервью и карьерный трек.

Реальные офферы — а не картинки из интернета

Реальные офферы с собеседований. Скрыты только компании, имена и контакты — суммы, должности и условия читаются, как в настоящих документах.

РЕКОРД

Работа

Тип контракта
Бессрочный трудовой договор

Испытательный срок
3 месяца

Зароботная плата
Два раза в месяц на ваш счёт

Отпуск
Ежегодный оплачиваемый отпуск 28 дней по ТК РФ

Размер заработной платы
Месечный доход после вычета НДФЛ: 400 000р + 20 000р
Итого: 420 000р
премия + 35% ежемесячная премия

567 000 Р/мес

АНОНИМИЗИРОВАНО

Предложение о работе в [компания]

Оклад
517 242 руб.
в месяц до вычета НДФЛ (gross)

Роль
Старший разработчик решений для [компания]

Привет, [имя],

Поздравляем!
Все этапы интервью успешно пройдены.

Приглашаем стать частью команды
Будем рады называть тебя коллегой, а не кандидатом.

Присоединяйся и делай цифровой мир безопаснее вместе с нами. Мы ждем тебя!

Чтобы сказать «да», подписи

517 000 Р/мес

АНОНИМИЗИРОВАНО

Понимание и реализация архитектуры развертывания:

- Проектирование CI/CD процессов;
- Управление инфраструктурой как кодом (Terraform/Helm);
- Обеспечение отказоустойчивости и масштабируемости.

Мониторинг и логирование:

- Настройка Prometheus, Grafana, ELK/Opensearch;
- Анализ логов и метрик, настройка алертов.

Безопасность и доступ:

- Настройка TLS/mTLS, RBAC, secrets;
- Управление доступом к сервисам и базам данных.

Настоящая позиция предлагается вам на следующих условиях:

Оплата труда:

Структура дохода на испытательном сроке и по результатам испытания:

- Размер квартального дохода на испытательном сроке и по результатам испытания: 1 308 000 рублей

Минимальный оклад: 436 000 руб. (срок испытания составит 436 000, из них: 34 000 руб. — ежемесячная премия МВО: 87 000 [20%])

436 000 Р/мес

АНОНИМИЗИРОВАНО

СТРОГО КОНФИДЕНЦИАЛЬНО

28 [имя] г.

Уважаемый [имя],

Компания ООО [компания] рада сделать Вам предложение о работе в должности старший разработчик облачной платформы Группы разработки облачной платформы Отдел разработки Департамента информационных технологий.

В круг Ваших обязанностей будет входить:

- Осуществлять разработку систем интеграции продуктов Общества;
- Осуществлять полный цикл жизни продукта от проработки продуктового релиза до его запуска на серверы;
- Осуществлять разработку новых сервисов продуктов Общества на языке [язык]

403 000 Р/мес

АНОНИМИЗИРОВАНО

12 [имя] г.

Предложение о сотрудничестве

Уважаемый [имя],

Компания ООО [компания] рада сделать Вам предложение о сотрудничестве на следующих условиях:

Выполнение задач DevOps-инженера на проекте: [компания]

Задачи на проекте:

- Разработка и сопровождение CI/CD
- Разработка и внедрение манифестов k8s/openshift
- Обеспечение повторяемости окружения
- Обеспечение стабильности автоматизации сборок приложений
- Разработка скриптов автоматизации рутинных задач администратора

Итого: 372 000 руб. (срок испытания составит 372 000, из них: 34 000 руб. — ежемесячная премия МВО: 87 000 [20%])

372 000 Р/мес

АНОНИМИЗИРОВАНО

Предложение о работе в [компания]

Будем рады видеть вас в нашей команде в качестве инженера.

Вы сможете участвовать в развитии продукта, которым пользуются более 30 миллионов человек каждый месяц. [компания] — самый популярный сайт объявлений в России, входит в международную медиагруппу [компания]

Подразделение: Trust and safety

Уважаемый [имя],

Оклад: 358 000 руб. gross

Итого: 358 000 руб. gross

Полный рабочий день.

358 000 Р/мес



Цифры настоящие — рекорд **567 000 Р/мес на руки** (оклад 420 000 + 35% премия). Это результат системной подготовки, что проходят ученики **BoostMentor**.



— СЛЕДУЮЩИЙ ШАГ

Ты увидел путь. Пора по нему пойти.

У тебя теперь есть roadmap, карта навыков и понимание, что спрашивают на собеседах. Это достаточно, чтобы начать — самому или с программой.



Что внутри программы

13 модулей, 123+ урока, реальные VM-стенды
Provision → break → fix → AI-ревью отчёта
1165+ вопросов · mock-интервью · карьерный трек
Capstone + ачивка **devops-interview-ready**
tg · @BOOOSTMENTOR · web · boostmentor.ru

ТОП-30 вопросов с реальных DevOps-собесов

Отобрал 30 вопросов, которые задают чаще всего и на которых чаще всего валятся — самые частые и самые каверзные. К каждому — **короткий ответ, который можно сказать вслух за 30–40 секунд**, ключевой тезис и ошибка, по которой интервьюер сразу видит новичка.

30 вопросов

8 тем

ответ за 30–40 секунд

формат • как на реальном собесе

КАРТА РАЗДЕЛА • ТЕМЫ И НОМЕРА ВОПРОСОВ

Linux

ВОПРОСЫ 1–6

6

Сети

ВОПРОС 7

1

Docker

ВОПРОСЫ 8–11

4

CI / CD

ВОПРОСЫ 12–14

3

Kubernetes

ВОПРОСЫ 15–21

7

Observability

ВОПРОСЫ 22–25

4

IaC • Terraform + Ansible

ВОПРОСЫ 26–27

2

Базы данных и секреты

ВОПРОСЫ 28–30

3

КАК УСТРОЕНА КАЖДАЯ КАРТОЧКА

ВОПРОС

Реальная формулировка — так спрашивают на собесе, без «учебной» полировки.

ОТВЕТ

Что сказать вслух. Коротко, по делу, с командами и терминами, которые ждёт интервьюер.

КЛЮЧЕВОЕ

Один тезис, который отличает практика от «читал в статье». Скажи его — и вопрос закрыт.

ОШИБКА

Самый частый провал на этом вопросе. Не повторяй.



КАК ПОЛЬЗОВАТЬСЯ

Закрой ответ и ответь вслух. Потом сверься: сказал ли ключевой тезис, не попал ли в ошибку. Где «плывёшь» — это и есть твоя карта пробелов: иди в соответствующую главу гайда и делай задание руками.



Вопросы 1–4 • страница 1 из 8

LINUX вопросы 1–6

01 Сервер тупит — куда смотреть?

LINUX

Иду по ресурсам, а не гадаю: `uptime` / `htop` — load average и топ по CPU; `free -h` — колонка **available**; `iostat -xh 1 3` — диск (%util, await); `df -h` и `df -i` — место и inode. Load average — длина очереди задач за 1/5/15 минут: сравниваем с числом ядер, а не с единицей.

✓ **КЛЮЧЕВОЕ** Процесс в состоянии D в `htop` ждёт диск, а не CPU.

✗ **ОШИБКА** Смотреть только CPU и сравнивать load с 1.0 вместо числа ядер.

02 Load average высокий, а CPU простаивает (load 12 на 4 ядрах, idle почти 100%) — почему и чем диагностировать?

LINUX

Значит, процессы стоят в очереди не на CPU, а на I/O: load считает и runnable, и D-state (uninterruptible sleep). Подтверждаю по iowait в `iostat -x` / `mpstat`, виновника ищу через `iostat -ao`, D-процессы — `ps -eo pid,stat,comm`, в `dmesg` — hung_task и ошибки диска. Частые причины: перегруженный диск, NFS-таймаут, RAID-ребилд.

✓ **КЛЮЧЕВОЕ** D-state = ждёт I/O. Load average — это не нагрузка CPU.

✗ **ОШИБКА** «CPU idle — значит всё в порядке» и не посмотреть iowait.

03 Удалили конфиг, а процесс его держит — можно восстановить?

LINUX

Да, пока процесс жив. `rm` убирает только запись в каталоге, а inode живёт, пока открыт хотя бы один дескриптор. Ищу `lsof | grep deleted` → PID и номер FD, копирую `cp /proc/<pid>/fd/<fd> /etc/restored.conf`. Место на диске тоже не освободится до закрытия FD — отсюда расхождение `df` и `du`.

✓ **КЛЮЧЕВОЕ** Файл жив в `/proc/<pid>/fd/`, пока процесс его не закрыл.

✗ **ОШИБКА** Перезапустить процесс «чтобы отпустил» — после этого восстанавливать уже нечего.

04 Сигналы: SIGTERM vs SIGINT vs SIGKILL — как процесс реагирует, как корректно остановить, когда kill -9?

LINUX

SIGTERM (15) и SIGINT (2, Ctrl+C) — просьба завершиться: процесс может перехватить, доделать работу и закрыть соединения. SIGKILL (9) перехватить нельзя — ядро убивает сразу, буферы и транзакции теряются. Порядок: SIGTERM → grace period → SIGKILL, если не вышел. Так работают `systemctl stop`, `docker stop` и Kubernetes.

✓ **КЛЮЧЕВОЕ** В контейнере PID 1 = `sh` не форвардит сигналы — ехес-форма или tini.

✗ **ОШИБКА** Сразу `kill -9`, не дав процессу завершиться корректно.

Вопросы 5–8 • страница 2 из 8

05 systemd: что происходит при загрузке Linux (от кнопки питания до login), что такое unit; как поднять node_exporter как сервис?

LINUX

POST/UEFI → GRUB → ядро + initramfs (драйверы до монтирования root) → `/sbin/init` = systemd (PID 1) → targets: `sysinit` → `basic` → `multi-user`. Unit — декларативный файл ресурса: `.service`, `.timer`, `.socket`, `.mount`, `.target`. `node_exporter`: unit в `/etc/systemd/system/`, `systemctl daemon-reload`, `enable --now`, метрики на `:9100/metrics`, логи — `journalctl -u`.

✓ **КЛЮЧЕВОЕ** После правки unit — `daemon-reload`, иначе изменения не подхватаются.

✗ **ОШИБКА** Пропустить `initramfs` и забыть `After=network.target` для сетевого сервиса.

06 Что такое cgroups и namespaces — что изолируют, что ограничивают? (отсюда же — контейнеризация vs виртуализация)

LINUX

Namespaces изолируют **видимость**: `pid`, `net`, `mnt`, `uts`, `ipc`, `user` — контейнер видит только своё дерево процессов, сеть и ФС. Cgroups ограничивают **потребление**: CPU, память, I/O, число процессов; `memory limit` в Kubernetes → `cgroup` → `OOMKill`. Контейнер = namespaces + cgroups + union-FS на общем ядре хоста; VM несёт своё ядро поверх гипервизора.

✓ **КЛЮЧЕВОЕ** Namespace — «что видно», cgroup — «сколько можно».

✗ **ОШИБКА** Считать контейнер изолированным как VM: уязвимость ядра общая для всех.

СЕТИ вопрос 7

07 Nginx: в каком порядке обрабатываются location-блоки; как сделать редирект HTTP → HTTPS; как валидировать конфиг перед reload?

СЕТИ

Порядок не «сверху вниз»: сначала точное `=` (стоп), затем самый длинный prefix — если у него `^~`, стоп; иначе `regex` `~` / `~*` в порядке файла, первый совпавший побеждает; если ни один — самый длинный prefix. Редирект в server на `:80`: `return 301 https://$host$request_uri`; . Перед reload — `nginx -t`.

✓ **КЛЮЧЕВОЕ** `return 301` дешевле `rewrite` — без `regex`-движка.

✗ **ОШИБКА** Reload без `nginx -t`: синтаксическая ошибка кладёт прод.

DOCKER вопросы 8–11

08 Чем контейнеризация отличается от виртуализации (контейнеры vs VM)?

DOCKER

VM — гипервизор эмулирует железо, внутри полноценная ОС со своим ядром: гигабайты, старт секунды, сильная изоляция. Контейнер — процесс на ядре хоста, изолированный namespaces и ограниченный cgroups: старт миллисекунды, образ мегабайты, изоляция слабее (общее ядро). Не конкуренты: кластер Kubernetes обычно живёт на VM.

✓ **КЛЮЧЕВОЕ** У контейнера нет своего ядра — это главное отличие.

✗ **ОШИБКА** «Контейнер — это лёгкая виртуальная машина».

Вопросы 9-12 · страница 3 из 8

09 ADD vs COPY; RUN vs CMD vs ENTRYPOINT — в чём разница?

DOCKER

COPY просто копирует из контекста; **ADD** ещё распаковывает tar и тянет URL — поэтому по умолчанию **COPY**. **RUN** выполняется при сборке и создаёт слой. **CMD** — команда по умолчанию, легко переопределяется в `docker run`. **ENTRYPOINT** — фиксированная точка входа, **CMD** становится её аргументами. Обе — в ехес-форме `["..."]`.

✓ **КЛЮЧЕВОЕ** Shell-форма запускает через `/bin/sh -c`: PID 1 = sh, SIGTERM до приложения не доходит.

✗ **ОШИБКА** **ADD** для обычного копирования; **CMD** вместо **ENTRYPOINT** у сервиса.

10 Что такое multi-stage build и зачем он нужен; «жирный» образ ~1 ГБ — как оптимизировать?

DOCKER

Несколько **FROM** в одном Dockerfile: в builder-стейдж компилятор и зависимости, в финальный через **COPY --from=builder** уезжает только артефакт. База финала — **distroless**, **alpine** или **scratch**. Ещё: **.dockerignore**, **apt-get install** и очистка кэша в одном **RUN**, редко меняющиеся слои выше — чтобы работал кэш.

✓ **КЛЮЧЕВОЕ** Go-сервис: builder на **golang** → **distroless/static** ~2 МБ.

✗ **ОШИБКА** Toolchain в прод-образе и отдельный **RUN** на каждую команду — образ растёт в 10-50 раз.

11 Go-бинарь собран в Ubuntu-стейдж multi-stage, скопирован в Alpine — не запускается. Почему?

DOCKER

Разные libc: Ubuntu — glibc, Alpine — musl. С **CGO_ENABLED=1** (по умолчанию) бинарь динамически линкован с glibc и в Alpine не находит библиотек: `no such file or directory` / `not found`. Решение — **CGO_ENABLED=0 go build**: статический бинарь работает где угодно, включая **scratch**.

✓ **КЛЮЧЕВОЕ** Проверка: `ldd ./service` → «not a dynamic executable».

✗ **ОШИБКА** Искать проблему в архитектуре amd64/arm64 или ставить glibc в Alpine.

CI / CD вопросы 12-14

12 Опиши свой идеальный CI/CD pipeline от начала до конца — какие stages и зачем каждый?

CI / CD

Хребет `lint → test → build → push → deploy`, каждый stage — gate: красный — дальше не едет. lint ловит грубое за минуту; test с coverage и `artifacts: reports: junit`; build multi-stage образа только после зелёных тестов; push с immutable-тегом по sha коммита, не `:latest`; между build и deploy — скан (Trivy); deploy в dev автоматом, в prod — `when: manual`.

✓ **КЛЮЧЕВОЕ** Immutable-тег = откат одной командой на предыдущий sha.

✗ **ОШИБКА** Один stage «сделать всё» и авто-деплой в прод без ручного гейта.

Вопросы 13–16 · страница 4 из 8

13 Спроектируй .gitlab-ci.yml: (1) автоматически создаёт MR при пуше ветки, (2) гоняет тесты, (3) постит результаты комментарием в MR.

CI / CD

Три job в `needs` -цепочке. `create-mr` : `curl POST /projects/:id/merge_requests` с токеном из masked CI-переменной, IID пишем в `artifacts: reports: dotenv` . `run-tests` : тесты + junit-отчёт. `post-comment` : читает IID из dotenv и постит `POST .../merge_requests/:iid/notes` . `workflow: rules` — только не-default ветки.

✓ **КЛЮЧЕВОЕ** Значение из job1 доступно job2 через artifacts, но не в `rules:` — они считаются на старте pipeline.

✗ **ОШИБКА** Токен прямо в YAML и парсинг ответа API через `grep` вместо `jq` .

14 С какими CI/CD-инструментами работал (GitLab CI / Jenkins / TeamCity) — что именно делал?

CI / CD

Отвечай конкретикой, не логотипами: какой раннер поднимал и с каким executor (shell — прост, но грязит хост; docker — изоляция; kubernetes — pod на job, автоскейл), какие stages, что нетривиального — кэш `node_modules` / Go-модулей по `CI_COMMIT_REF_SLUG` , environments и review apps на MR, protected branches для прода, artifacts между stages, что чинил.

✓ **КЛЮЧЕВОЕ** Секреты — только в masked / protected CI-переменных.

✗ **ОШИБКА** «Знаю GitLab CI» без примера, что именно настраивал руками.

KUBERNETES вопросы 15–21

15 Из каких компонентов состоит кластер и что пошагово происходит при `kubectl apply -f deployment.yaml`?

KUBERNETES

`kubectl` шлёт манифест в kube-apiserver → authn → authz (RBAC) → admission (mutating, затем validating) → запись в etcd. Deployment-controller в controller-manager создаёт ReplicaSet, тот — Pod с пустым `nodeName` . Scheduler выбирает ноду по requests. kubelet на ноде видит под через watch, вызывает CRI (containerd), CNI настраивает сеть, статус уходит обратно в API.

✓ **КЛЮЧЕВОЕ** Никто, кроме apiserver, не ходит в etcd — kubelet тоже читает через API.

✗ **ОШИБКА** «kubelet берёт под из etcd» и «Docker как runtime» (убран в 1.24).

16 Как выглядит отказоустойчивый (HA) Kubernetes-кластер и как его реализовать?

KUBERNETES

Нечётное число control-plane нод (3 или 5) — кворум etcd по Raft: три переживают потерю одной. Перед API-серверами — балансировщик, сам отказоустойчивый: HAProxy + Keepalived с VIP на bare-metal или облачный LB. Воркеры по зонам, PodDisruptionBudget для критичных сервисов, регулярный `etcdctl snapshot save` с проверкой restore.

✓ **КЛЮЧЕВОЕ** При потере control plane поды продолжают работать — kubelet автономен, ломаются только изменения.

✗ **ОШИБКА** Две или четыре etcd-ноды и балансировщик без HA — новая точка отказа.

Вопросы 17–20 · страница 5 из 8

17 Какие типы Kubernetes Service существуют и для чего каждый (ClusterIP / NodePort / LoadBalancer / Headless)?

KUBERNETES

ClusterIP — виртуальный IP только внутри кластера, дефолт для сервис-к-сервису. NodePort — плюс порт 30000–32767 на каждой ноде. LoadBalancer — плюс внешний IP от облака или MetalLB. Headless (`clusterIP: None`) — без VIP: DNS отдаёт IP подов напрямую; нужен StatefulSet и gRPC-клиентам с балансировкой на своей стороне.

✓ **КЛЮЧЕВОЕ** LoadBalancer — L4, Ingress — L7 HTTP-роутинг: разные слои.

✗ **ОШИБКА** Ждать от NodePort нормальный порт 80/443.

18 Чем отличаются Deployment / StatefulSet / DaemonSet / ReplicaSet — когда что использовать?

KUBERNETES

ReplicaSet держит N одинаковых подов — внутренний примитив. Deployment управляет ReplicaSet-ами: rolling update и `rollout undo` — для stateless. StatefulSet даёт стабильные имена `app-0`, `app-1`, свой PVC у каждого пода и упорядоченный старт — для БД, Kafka, etcd. DaemonSet — ровно один под на каждой ноде: node-exporter, log-shipпер, CNI.

✓ **КЛЮЧЕВОЕ** PVC StatefulSet не удаляется вместе с ним — защита данных.

✗ **ОШИБКА** База данных в Deployment — без стабильного хранилища и идентичности.

19 Как внешний трафик попадает в кластер — от интернета до подов ingress-контроллера внутри кластера?

KUBERNETES

DNS → IP облачного LB, VIP (MetalLB / Keepalived) или NodePort ноды → Service ingress-контроллера → под ingress-controller (nginx / Traefik) — обычный Deployment, который читает Ingress-ресурсы и по `Host` / `path` выбирает backend Service → kube-proxy DNAT-ит ClusterIP в IP пода (iptables / IPVS). TLS terminates на LB или на контроллере (cert-manager).

✓ **КЛЮЧЕВОЕ** Ingress-ресурс — правила; Ingress Controller — процесс, который их исполняет.

✗ **ОШИБКА** Путать Ingress и Ingress Controller; думать, что DNS указывает на IP пода.

20 Три главных компонента control plane; какой компонент планирует поды; scheduler смотрит на requests или limits?

KUBERNETES

kube-apiserver — единственная точка входа и единственный писатель в etcd; etcd — source of truth (Raft, нечётное число нод, чувствителен к медленному диску); controller-manager — контроллеры в reconcile-цикле; kube-scheduler планирует поды. Scheduler смотрит только на `resources.requests` — это «броня» на ноде. Limits работают в рантайме: CPU throttling, память — OOMKill (exit 137).

✓ **КЛЮЧЕВОЕ** Scheduler — requests; limits — только рантайм.

✗ **ОШИБКА** «Scheduler смотрит на limits»; kubelet и kube-proxy в списке control plane.

Вопросы 21–24 · страница 6 из 8

21 Что такое sidecar-контейнер и чем отличается от init-контейнера?

KUBERNETES

Init-контейнеры выполняются до основных, последовательно, каждый должен завершиться с exit 0: дождаться БД, накатить схему, подготовить конфиг. Sidecar живёт параллельно весь срок пода, делит с ним сетевой namespace (localhost) и volumes: log-shipпер, Envoy service-mesh, Vault Agent. С 1.29 sidecar объявляется в `initContainers` с `restartPolicy: Always`.

✓ **КЛЮЧЕВОЕ** Sidecar слушает на localhost основного контейнера — сеть общая.

✗ **ОШИБКА** Называть init-контейнер sidecar-ом — самая частая путаница.

OBSERVABILITY вопросы 22–25

22 Как был устроен ваш мониторинг — инструменты, как собирались метрики, какие алерты настраивали?

OBSERVABILITY

Называй стек и зоны ответственности: kube-prometheus-stack (Prometheus Operator, node-exporter, kube-state-metrics, Alertmanager, Grafana) в Kubernetes; на BM — node-exporter и экспортеры. Scrape через ServiceMonitor / PodMonitor, алерты как код в PrometheusRule в Git, routing и grouping в Alertmanager (critical → дежурный, warning → чат), long-term — VictoriaMetrics / Thanos.

✓ **КЛЮЧЕВОЕ** Алерты по SLO / burn-rate дают меньше шума, чем пороги.

✗ **ОШИБКА** Нет grouping / inhibition — на инциденте лавина алертов; дашборды «руками в UI».

23 Какой стек логирования и мониторинга использовали?

OBSERVABILITY

Метрики — Prometheus или VictoriaMetrics (drop-in, лучше сжатие) + Grafana. Логи — PLG: Promtail / Fluent Bit → Loki → Grafana (LogQL), либо ELK / EFK с Elasticsearch и Kibana. Fluent Bit легче Fluentd; агент — DaemonSet, обогащает логи namespace / pod. Приложение пишет структурированный JSON — шиппер не парсит regex.

✓ **КЛЮЧЕВОЕ** Loki индексирует только лейблы, не текст: дёшево, но без full-text поиска.

✗ **ОШИБКА** Regex-парсинг в Logstash вместо JSON у источника.

24 Чем отличаются Histogram и Summary в Prometheus; rate() vs increase() — для каких метрик?

OBSERVABILITY

Histogram — приложение считает бакеты, квантиль вычисляет Prometheus через `histogram_quantile()`, поэтому бакеты складываются по репликам. Summary — квантили считает само приложение, агрегировать между репликами нельзя (среднее перцентилей бессмысленно). `rate()` — скорость в секунду для throughput; `increase()` — прирост за окно для счётных алертов.

✓ **КЛЮЧЕВОЕ** Для multi-replica сервиса почти всегда Histogram.

✗ **ОШИБКА** `irate()` в алертах — слишком чувствителен к спайкам.

Вопросы 25–27 · страница 7 из 8

25 Что такое высокая кардинальность лейблов в Prometheus и почему это проблема; примеры плохих и хороших лейблов?

OBSERVABILITY

Каждая уникальная комбинация значений лейблов — отдельный time series, и все активные серии живут в RAM Prometheus. Лейбл `user_id` при миллионе пользователей — миллион серий и OOM. Плохо: `user_id`, `request_id`, `email`, `IP`, полный URL с query. Хорошо: `method`, `status`, `service`, `env`, route-шаблон `/users/:id`.

✓ **КЛЮЧЕВОЕ** Следить за `prometheus_tsdb_head_series`.

✗ **ОШИБКА** Считать кардинальность проблемой диска — страдает память.

IAC • TERRAFORM + ANSIBLE вопросы 26–27

26 Что такое Terraform statefile и для чего он; где хранить (remote backend) и что такое state locking?

IAC

`terraform.tfstate` — JSON-маппинг ресурсов из HCL на реальные объекты (ID, атрибуты); по нему `plan` считает diff. В команде — remote backend (S3, GCS, Terraform Cloud) плюс state locking (DynamoDB для S3), иначе два одновременных `apply` портят state. Ритуал: `init` → `plan` → `apply`.

✓ **КЛЮЧЕВОЕ** State содержит секреты — не в Git, доступ по IAM.

✗ **ОШИБКА** Локальный state в команде и `tfstate` в репозитории.

27 Ansible: shell vs command — когда какой; что такое serial (rolling restart)?

IAC

`command` запускает бинарник напрямую — без pipe, redirect и `$VAR`, зато без shell-injection; `shell` идёт через `/bin/sh -c` и умеет всё это. Предпочтение — модулям `copy`, `template`, `service`: они идемпотентны. `serial` — размер батча хостов: play прогоняется по 1, по 25% или списком — rolling restart без общего даунтайма.

✓ **КЛЮЧЕВОЕ** `serial` — батч хостов в play; `forks` — параллелизм SSH-соединений.

✗ **ОШИБКА** `shell` везде вместо модулей и ожидание pipe в `command`.



ОБЩИЙ ПРИЁМ НА ЛЮБОЙ ВОПРОС

Отвечай по одной структуре: **что это** → **зачем** → **как проверю или что сделаю руками** → **где обычно ломается**. Интервьюер слышит не факты, а то, как ты думаешь. Скажи главное за 30–40 секунд — детали добавишь, если попросят глубже.



Вопросы 28–30 · страница 8 из 8

БАЗЫ ДАННЫХ И СЕКРЕТЫ вопросы 28–30

28 Какой у вас опыт с базами данных — с какими работали и что именно делали?

БАЗЫ ДАННЫХ

Структура: СУБД → что именно делал как DevOps. PostgreSQL: Patroni + etcd (3 ноды), streaming replication и `pg_stat_replication`, бэкапы pgBackRest / WAL-G в S3 с PITR и тестом restore, PgBouncer, мажорные апгрейды. Redis: RDB / AOF, Sentinel (failover) vs Cluster (шардинг). ClickHouse: шарды + реплики + Keeper. Плюс мониторинг: slow queries, vacuum, лаг реплик.

✓ **КЛЮЧЕВОЕ** Бэкап, HA и мониторинг — зона DevOps, а не только DBA.

✗ **ОШИБКА** Перечислить СУБД без того, что конкретно с ними делал.

29 PostgreSQL HA: что такое Patroni и split-brain; мастер упал в одном ДЦ — ваши действия?

БАЗЫ ДАННЫХ

Patroni держит leader-key с TTL в DCS (etcd / Consul); мастер продлевает его heartbeat-ом. Потерял связь с etcd — сам перестаёт принимать writes, не дожидаясь TTL. Когда ключ истёк, реплика с максимальным LSN забирает lock через CAS и промоутируется — split-brain невозможен, lock один. Действия: `patronictl list`, убедиться в failover, старый мастер вернуть через `pg_rewrite`, проверить лаг и приложения.

✓ **КЛЮЧЕВОЕ** От split-brain защищает lock в etcd, а не «кворум реплик».

✗ **ОШИБКА** Путать роли Patroni и etcd; не знать `patronictl`.

30 Бэкап PostgreSQL: чем логический отличается от физического; зачем бэкап, если уже есть репликация?

БАЗЫ ДАННЫХ

Логический (`pg_dump`) — SQL / custom-дамп: переносим между версиями, можно восстановить одну таблицу, но restore медленный и нет PITR. Физический (`pg_basebackup`, pgBackRest, WAL-G) — бинарная копия data dir: быстрый restore, та же мажорная версия, основа PITR через непрерывный WAL-архив. Репликация — не бэкап: `DROP TABLE` уедет на реплику мгновенно.

✓ **КЛЮЧЕВОЕ** Бэкап, который ни разу не восстанавливали, — не бэкап.

✗ **ОШИБКА** Только `pg_dump`, бэкап на том же сервере, «репликация нас спасёт».

ЭТО ТОП-30 • ДАЛЬШЕ — ВСЬ БАНК

Ещё 34 вопросов и 56 задач — с полными разборами

Разборы уровня senior, ключевые тезисы, типовые ошибки и практические задачи по каждой теме — бесплатно на boostmentor.ru/devops. А если хочешь не список, а маршрут с практикой и обратной связью — пиши мне.

БАНК ВОПРОСОВ boostmentor.ru/devops
НАПИСАТЬ МНЕ t.me/viktor_golang



ВСЕ РАЗБОРЫ